HACK WITH RASPBERRY PI COMMUNITY

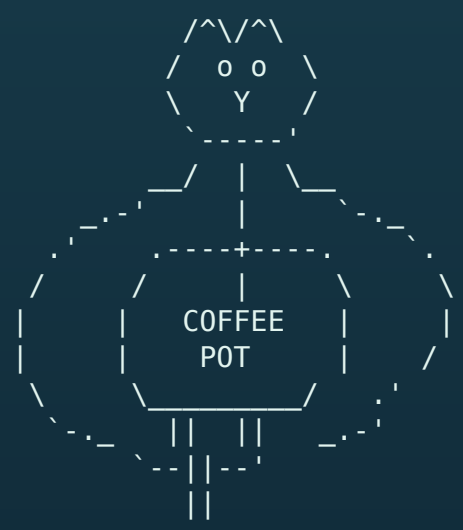
Raspberry Pi

for Technical Support Operatives in Africa

A field guide to Raspberry Pi, Linux, and responsible support

THE COFFEE POT SNAKE

ORIGINAL ASCII ART



William Majanja

An initiative of the Hack with Raspberry Pi community

FIELD GUIDE · OFFLINE-FRIENDLY · OPEN FORMATS

- Raspberry Pi for Technical Support Operatives in Africa
  - by William Majanja
  - Start here
  - What the guide covers
  - Design and reading experience
  - Authoring and verification environment
  - Kali Linux and CPIP-
  - Historical material
  - Responsible use, credits, and licensing
  - Contributing
- Responsible Use
  - Before a security test
  - Treat commands as actions
  - Platform boundaries
  - Project-specific cautions
- Code of Conduct
  - Expected conduct
- Contributing
  - Documentation changes
  - Testing a contribution
  - Pull requests
- Licensing and Provenance
  - Reuse guidance
- Security Policy
  - Reporting a problem in this guide
- Legacy Kali Tool Material
  - Archived chapter indexes
- Introduction
  - Who this guide is for
  - What is covered
  - Operating-system choices
  - Authoring environment and test scope
  - Zero-dependency reading and challenges
- Guide Structure
- Linux Topics Now Covered
- William Majanja — Informatics Portfolio
  - Selected work
    - The Muainishi series
  - Peace Operations Training Institute
  - Historical security artifact: Jester-themed ransomware fork
  - Profiles and attribution
- Contributors and Historical Attribution
  - Original edition
  - Current edition
- Acknowledgments
- References and Credits
  - Raspberry Pi and Linux support
  - Kali Linux (optional lab platform)
  - Recipe-format references
  - Peace operations, humanitarian protection, and standards

    - Protocol and learning-project references
    - Public health, agriculture, and corporate accountability
    - Attribution and reuse
- Raspberry Pi Field Support
    - Field-support path
    - Keep the platform identity clear
    - Test scope
    - Primary references
- Imaging and First Boot
    - 1. Identify the deployment before writing an image
    - 2. Choose the operating system
    - 3. Prepare boot media carefully
    - 4. First boot checklist
    - 5. Headless network setup
    - 6. Power, shutdown, and hardware safety
    - 7. References
- Support and Troubleshooting Playbooks
    - 1. Capture a safe baseline
    - 2. Device does not boot
    - 3. Device boots but has no network
    - 4. Remote access is unavailable
    - 5. Desktop or display problem
    - 6. Slow, hot, or unstable device
    - 7. Service failure
    - 8. Storage or update problem
    - 9. Escalation notes
- Field Operations, Maintenance, and Handover
    - 1. Build a supportable deployment
    - 2. Updates and change control
    - 3. Backups and recovery
    - 4. Data protection and localization
    - 5. Handover template
    - 6. Hardware handling
    - 7. Authoritative references
- Linux Foundations
    - Chapters
    - Platform tested for this edition
- Linux Kernel and System Model
    - 1. What the kernel does
    - 2. Boot and login at a high level
    - 3. Identify the running system
    - 4. Processes, memory, devices, and virtual filesystems
    - 5. Drivers and modules
    - 6. Kernel updates and distribution differences
    - 7. Kernel configuration and tuning
    - 8. Building a kernel: when and where
    - 9. Security concepts
    - 10. References
- Bash Shell Programming
    - 1. Commands, arguments, and exit status

    - 2. Quoting and expansion
    - 3. Variables and environment
    - 4. Redirection and pipelines
    - 5. Globs, arrays, and loops
    - 6. Conditions and functions
    - 7. A small, safe script
    - 8. Safe scripting practices
    - 9. Bash versus other shells
    - 10. References
- Practical Command-Line Recipes
    - Quick recipe index
    - 1. Identify the distribution, architecture, and kernel
    - 2. Recall a shell command without blindly rerunning it
    - 3. Format tab-separated output as a table
    - 4. Count repeated lines
    - 5. Read a file one line at a time in Bash
    - 6. Inspect disk usage
    - 7. Check local listening sockets
    - 8. Read service status and logs
    - 9. Inspect network configuration without scanning
    - 10. Check a package before installing it
    - 11. Update Kali deliberately
    - 12. Compare a local file with an authorized remote copy
    - 13. Verify a downloaded file against its publisher’s checksum
    - 14. Preview an archive before extracting it
    - 15. Serve a local directory only to this computer
    - 16. Capture a terminal session carefully
    - 17. Useful command-line habits
    - Command Index
    - Source and editorial note
- Peace Operations Field Support
    - Chapters
    - High-impact practices covered
    - Read the primary sources
- Mine Action and Explosive-Hazard Support
    - 1. Why mine action matters to technical support
    - 2. High-impact field practices
        - A. Start with non-technical survey and evidence quality
        - B. Make risk education local, inclusive, and feedback-driven
        - C. Treat victim assistance as a coordinated, long-term responsibility
        - D. Coordinate nationally and across sectors
        - E. Use technology as a support layer—not a substitute for expertise
    - 3. A support workflow for mine-action information
        - EORE material release checklist for support staff
    - 4. Explosive-hazard encounters and role boundaries
    - 5. Sources
- Child Protection and Women, Peace and Security
    - 1. Child protection in armed conflict
        - What technical teams should do
    - 2. Women, Peace and Security (WPS)

        - Applying WPS principles to technical support
    - 3. Resolution and mandate awareness
    - 4. Further references
- Selected Security Council Resolution Reference
    - Mine action and explosive hazards
    - Children and armed conflict
    - Women, Peace and Security
    - Related protection-of-civilians references
    - How to use this reference
        - Official starting points
- Informatics, Public Health, and Corporate Accountability
    - 1. The Muainishi application series
        - A practical assessment checklist
    - 2. Corporate incentives and documented public harm
        - Documented examples—not claims about every company
    - 3. Genetically engineered crops, pesticides, and cancer claims
    - 4. A reproducible method for investigating harm
        - Other sectors to examine with the same method
    - 5. How Raspberry Pi and open Informatics can help
    - References
- Zero-Dependency Quizzes and CTF-Style Challenges
    - Zero-dependency rules
    - How to play
    - Base Levels — Field Support
        - BASE-0 — Identify the box
        - BASE-1 — Protect the data before recovery
        - BASE-2 — Diagnose one layer at a time
        - BASE-3 — Protect sensitive records
    - Quiz Levels — Knowledge Check
        - QUIZ-0 — Who handles a suspected explosive item?
        - QUIZ-1 — Which resolution is the thematic Security Council reference on mine action?
        - QUIZ-2 — What is the safe treatment of a child-protection report in a support ticket?
        - QUIZ-3 — What does meaningful WPS participation require in support-system design?
        - QUIZ-4 — How should you assess the claim “genetically engineered crops cause cancer”?
        - QUIZ-5 — Does a company’s profit motive prove that it intended to cause a particular harm?
    - Crypto Flag Levels
        - CRYPTO-0 — ROT13 is encoding
        - CRYPTO-1 — Caesar shift is a toy
        - CRYPTO-2 — Hashes are not ciphertext
    - Debrief and next steps
    - Inspiration and attribution
- Challenge Answer Key
    - Base Levels — Field Support
        - BASE-0
        - BASE-1
        - BASE-2
        - BASE-3
    - Quiz Levels
    - Crypto Flag Levels
        - CRYPTO-0

        - CRYPTO-1
        - CRYPTO-2
- CPIP-: Coffee Pot Internet Protocol
    - What “zero dependency” means here
    - The project’s stated design goals
    - Guide to this chapter
    - Safe starting point
    - Primary sources
- CPIP- Architecture and Principles
    - 1. The protocol joke and the project
    - 2. Architecture as described by the project
    - 3. Reading the design goals critically
        - RFC fidelity
        - Multi-transport resilience
        - Cover traffic and privacy
        - Active network defense
        - Cryptographic claims
    - 4. Threat model and data questions
    - 5. Source/version discipline
- CPIP-: Local-Only Setup
    - 1. Check prerequisites and source
    - 2. Start a local-only service
    - 3. Verify a basic local request
    - 4. CLI and dashboard
    - 5. Optional transports and hardware
    - 6. What has and has not been tested
- CPIP-: Operations and Security Review
    - 1. Configuration inventory
    - 2. Network exposure and authentication
    - 3. Keys, messages, and persistence
    - 4. Crypto language: precise claims
    - 5. Active defense and diagnostics
    - 6. Deployment modes
    - 7. Troubleshooting checklist
    - 8. Shutdown and cleanup
    - 9. References

Raspberry Pi for Technical Support Operatives in Africa

by William Majanja

An initiative of the **Hack with Raspberry Pi** community.

An open-source, zero-dependency field guide to supporting Raspberry Pi computers and Linux systems. It combines practical setup, troubleshooting, Bash recipes, security fundamentals, peace-support data responsibilities, and carefully scoped Kali/Linux material.

This guide is designed for technical support operatives working across Africa's varied deployment environments. It does not assume that every site has the same power, connectivity, language, hardware supply, or support capacity; use the checklists to document local constraints and plan accordingly.

> **Unofficial:** this book is not produced, endorsed, or maintained by Raspberry Pi Ltd, Debian, Kali Linux, or OffSec. Use security tools only on systems and networks you own or have explicit authorization to assess.

Start here

- **Download the book:** PDF for print and offline reading · EPUB for e-readers · Coffee Pot Snake cover
- Book map and reading paths
- Introduction and scope
- Raspberry Pi field-support guide
- Linux foundations: kernel, Bash, and command-line recipes
- Peace Operations field-support context
- Informatics, public health, and corporate accountability
- Zero-dependency quizzes and CTF-style challenges
- CPIP- user guide
- William Majanja's Informatics portfolio

What the guide covers

- **Raspberry Pi support:** model-aware setup, OS imaging, first boot, connectivity, troubleshooting, safe remote administration, updates, backups, and handover.
- **Linux foundations:** the kernel and user-space model, Bash scripting, and practical command-line recipes with explanations and checks.
- **African field operations:** offline-friendly procedures, locally adaptable communication and training, reliable recovery, privacy, and site-specific constraints without assuming one context fits all.
- **Peace-support and humanitarian context:** safe technical support for mine-action information systems, child-protection data, WPS participation, and relevant UN Security Council frameworks. This is not EOD, peacekeeper, or child-protection operational training.
- **Public-interest Informatics:** the Muainishi application series, health-data safeguards, corporate-accountability evidence, and careful distinctions between GE crops, specific traits, pesticide exposures, cancer hazards, and exposure-specific risks.

- **Security learning:** authorization, least privilege, isolated labs, source verification, and a historical, unreviewed Kali-tool archive kept separate from maintained guidance.
- **Self-study challenges:** Base and Crypto tracks with quiz prompts and flags, solvable from the book and standard shell tools without installing challenge software.
- **CPIP- and Informatics projects:** a version-aware guide to William Majanja's Coffee Pot Internet Protocol and selected projects from both GitHub accounts.

The book is **zero-dependency to read and browse**: it is plain Markdown on GitHub, with no build system, required account, or site generator. The software and devices discussed in the guide may have their own dependencies.

Design and reading experience

The book uses task-based recipes, progressive reading paths, topic categories, plain-language explanations, and source/license notes. This design takes inspiration from the local *Command Line Kung Fu* recipe reference and from Babe's Bookstore, which describes itself as a category-based digital library with rights information. No text, code, books, or assets are copied from either source; the bookstore's catalogue and rights claims are not independently audited here.

Authoring and verification environment

The authoring desktop is **Debian GNU/Linux 13 (Trixie), ARM64, KDE Plasma 6.3.6, Wayland**. The kernel identifies Raspberry Pi ARM hardware, but this does not mean every model, OS image, peripheral, or field deployment has been tested. Kali-specific commands and Raspberry Pi OS features are labeled and must be checked on their own target platforms before being called tested.

Kali Linux and CPIP-

Kali appears as a **specialized, optional security-lab platform**, not as the default operating system for routine Pi support. Check Kali's ARM documentation for model and image compatibility. Never mix Kali package repositories into Debian or Raspberry Pi OS.

CPIP- is William Majanja's independent Coffee Pot Internet Protocol project, not an official Raspberry Pi, Debian, or Kali component. Its guide labels dependencies and source-stated security claims; it does not treat those claims as independently validated.

Historical material

Older Kali-tool folders are retained as an **unreviewed historical archive**, not as current tool recommendations. Empty placeholders have been removed; surviving pages may contain obsolete commands, exploit/persistence examples, broken links, or third-party content. Browse the archive note before opening those folders. The curated contents list identifies the maintained reading path.

Responsible use, credits, and licensing

Use a disposable VM or isolated lab for hands-on security work. Do not expose experimental services to a network, run unknown code with elevated privileges, or test against a third party without written authorization. See Responsible Use, Code of Conduct, Security Policy, and Licensing and Provenance.

The original edition’s attribution to Chandrapal and Hack with Github is preserved separately from this edition. The existing CC0 file is inherited; it does not relicense third-party content.

Contributing

Corrections and well-sourced improvements are welcome. Read CONTRIBUTING.md before submitting changes.

Responsible Use

This guide is for Raspberry Pi and Linux technical support, peace-support information systems, and authorized security learning. A tool's presence in Kali does not grant permission to use it against a system.

Before a security test

1. Obtain written authorization from the system owner.
2. Define exact systems, accounts, network ranges, time windows, and allowed methods.
3. Use a disposable VM or isolated lab for examples and unfamiliar software.
4. Agree on rate limits, data handling, stop conditions, contacts, and reporting before testing.
5. Keep evidence and logs secure; avoid collecting data outside the agreed scope.
6. Stop if the work could disrupt a service or affect a system not explicitly in scope.

Do not use example targets, public IPs, wireless networks, accounts, or devices as practice systems unless you own them or have explicit authorization. Do not use this guide to evade monitoring, intercept someone else's traffic, maintain unauthorized access, deploy malware, or conceal activity.

Treat commands as actions

Read each command before running it. Pay special attention to:

- `sudo`, root shells, package-source changes, and service enablement;
- redirection operators (>, >>), recursive file operations, and paths supplied by variables;
- network listeners, port forwards, scanning options, and commands that contact remote hosts;
- scripts downloaded from the internet or copied from an old book;
- commands that handle passwords, keys, personal data, or sensitive logs.

Prefer inspection first, least privilege, clear output paths, and a tested rollback. A checksum is useful only when compared with an authentic checksum from the publisher. A self-signed TLS certificate or an algorithm name is not proof that a service is secure.

Platform boundaries

Kali and Debian are related but not interchangeable. Use the package repositories, update instructions, and service guidance for the distribution actually installed. **Never add Kali repositories to Debian as a shortcut.** The Debian/KDE/Wayland environment used to author this guide is not a substitute for testing Kali-specific packages or workflows.

Project-specific cautions

- The CPIP- chapter describes an independent research project. Keep initial tests loopback-only and treat the project's security claims as unverified unless independently reviewed.
- The Jester-themed ransomware fork in the author portfolio is included only as a historical security artifact. Do not build or run it on a personal or production system.
- Health, election, land, and agricultural Informatics projects linked from the portfolio are described according to their repositories; this book does not claim clinical, electoral, legal, or operational validation.

- Mine-action, child-protection, and WPS content is an orientation for technical support roles. It is not EOD, mine-clearance, peacekeeper, child-protection, or gender-adviser training. Follow the current mandate, national standards, qualified specialists, and organizational safeguarding rules.
- Never approach, touch, identify, move, mark, or attempt to disable a suspected explosive item. Use locally approved emergency reporting procedures and trained authorities.

Code of Conduct

This project is part of the Hack with Raspberry Pi community. We aim to make technical learning respectful, safe, and welcoming to people with different backgrounds, experience levels, languages, abilities, and access to equipment.

Expected conduct

- Be respectful and constructive; criticize ideas and documentation, not people.
- Make space for questions and different levels of experience.
- Respect privacy, confidentiality, and cultural context. Do not publish another person’s data, contact details, or field information without consent.
- Keep technical examples within authorized, safe, and clearly described scope.
- Give accurate credit and follow source licenses.

Harassment, discrimination, threats, doxxing, and deliberate unsafe conduct are not acceptable. If a concern arises, contact the repository maintainer through an appropriate private channel or report mechanism. Project maintainers may edit or remove content that violates this code or creates an immediate risk.

Contributing

Contributions that improve accuracy, clarity, accessibility, and responsible use are welcome.

Documentation changes

- Write in clear, direct English and explain unfamiliar terms on first use.
- Use the recipe structure: task, platform/prerequisites, command, explanation, verification, and safety/rollback notes.
- Prefer primary sources: official Kali or Debian documentation, upstream manuals, RFCs, standards, and the project's own source.
- Date platform-specific checks and name the distribution release, architecture, and software version tested.
- Label commands as **read-only**, **changes local state**, or **sends network traffic** when the effect is not obvious.
- Use fictional names, synthetic data, loopback, and isolated lab examples. Never include secrets, live credentials, or personal data.
- Clearly distinguish project claims, source-code observations, and independently verified results.
- Preserve historical authorship and source citations. Do not copy cookbook text or third-party project documentation into this repository without permission and compatible licensing.

Testing a contribution

This is a static Markdown guide and has no required build system. Check local links and navigation before submitting. If you add a Bash snippet, run `bash -n` on it and execute only harmless examples in a disposable directory. Do not test network or security examples outside an authorized lab.

Run the repository's optional dependency-free checker with `python3 tools/check_docs.py`. It uses Python's standard library and Bash syntax checking; it does not install packages or execute the examples.

For platform-specific commands, say exactly where they were tested. Do not describe a Debian test as a Kali test or imply that a linked third-party application was validated when it was not.

Pull requests

Keep changes focused. Summarize the learning outcome, list pages changed, cite sources, and note the checks performed. For large proposed chapters, open an issue first so scope and source provenance can be discussed.

Licensing and Provenance

The repository includes an inherited `LICENSE` file containing the text of CC0 1.0 Universal. The original tree also contains historical contributions and references to tools and projects by other authors. A license file in a repository does not, by itself, establish that its uploader owned every included work or that third-party code, prose, images, or documentation is covered by the same terms. The refreshed guide links to sources and writes original summaries; it does not copy material from the referenced books.

Reuse guidance

- The existing `LICENSE` is preserved as inherited project metadata; this edition does not claim to have verified the authority of every historical rights holder.
- Newly written pages link to upstream sources and summarize them in original language instead of reproducing their text.
- Third-party repositories, course material, images, code, and books retain their own licenses and attribution requirements.
- International Mine Action Standards are UN-owned and copyrighted; the cited IMAS pages state their reuse conditions. POTI training materials may be provided under revocable licenses. This guide links to those sources and summarizes themes in original language; do not copy their course pages, PDFs, scenarios, figures, or standards text into the repository.
- Fork status is identified where known. A public repository is not automatically open-source, and a fork is not sole authorship.
- Before redistributing legacy excerpts or assets, verify their source, license, and required attribution with the rights holder or upstream project.

The author portfolio is a directory of links and brief factual descriptions, not a copy of the linked projects. One linked project, `atlaz`, describes itself as proprietary; the Jester-themed ransomware repository is a fork and reports no license in GitHub metadata.

Security Policy

This repository is a documentation project; it does not ship a security application. Security issues in a linked upstream tool should be reported to that project through its own published disclosure channel, not attributed to this guide.

Reporting a problem in this guide

For a privacy concern, accidentally exposed secret, unsafe command, or other security-sensitive documentation issue, do not post the sensitive material in a public issue. Contact the repository maintainer through the private reporting option on GitHub if enabled. Otherwise, open a minimal public issue that describes the affected page without including secrets or exploit details, and ask for a private channel.

This repository has no published response-time guarantee. If an issue affects a third-party project, contact that project’s maintainers directly.

Legacy Kali Tool Material

The original book was a Kali-tool catalogue assembled around the Kali 1.x/2.x period. Its surviving pages are preserved as historical source material; they have not been reviewed for current package availability, command accuracy, safety, or third-party license compatibility. They are **not part of the maintained Raspberry Pi support curriculum**.

During this refresh, **276 empty Markdown placeholders were removed** because they contained no article text and falsely suggested complete coverage. The old chapter folders remain as historical indexes. In the current tree, substantive legacy article files remain only in the exploitation-tools folder, one information-gathering entry, and the Awesome Lists page. Some of these old articles discuss exploitation, persistence, social engineering, or other dual-use techniques. Do not treat the presence of a page as a recommendation or run its commands on a system.

Archived chapter indexes

- Exploitation Tools
- Forensics Tools
- Hardware Hacking
- Information Gathering
- Maintaining Access
- Password Attacks
- Reporting Tools
- Reverse Engineering
- Sniffing and Spoofing
- Stress Testing
- Web Applications
- Wireless Security
- External Security Resources

For current Kali commands and supported packages, use the official Kali documentation and the relevant upstream project. For reuse of archived material, consult Licensing and Provenance and verify the original source and rights holder.

Introduction

Raspberry Pi for Technical Support Operatives in Africa is a field-oriented, source-linked guide by William Majanja. It is an initiative of the **Hack with Raspberry Pi** community.

The guide helps support staff understand, prepare, troubleshoot, maintain, and hand over Raspberry Pi and Linux systems. It combines task-first command-line recipes with underlying Linux concepts, platform notes, security practice, and protection-aware data support. The recipe format was informed by Jason Cannon's local *Command Line Kung Fu* reference and the category-based reader experience of Babe's Bookstore. All text and examples in this guide are independently written.

Who this guide is for

- Technical support operatives supporting Raspberry Pi computers and Linux systems.
- Trainers and learners who need practical command-line, troubleshooting, and recovery skills.
- Organizations using small computers in community, education, health, agriculture, civic, or field deployments.
- Security learners who want to distinguish normal technical support from authorized assessment work.
- Informatics practitioners evaluating health, agricultural, and public-interest data systems.

The title's regional focus is an invitation to adapt the material to African deployment contexts—not an assumption that every African site has the same infrastructure, language, climate, connectivity, power, or support resources. Document the conditions at each actual site and involve the people who use and maintain the systems.

What is covered

The field-support chapters cover board-aware imaging and first boot, network and desktop triage, Linux kernel and Bash foundations, practical command-line recipes, change control, backup and restore planning, privacy, and handover. The guide also introduces safe technical support for peace operations and humanitarian data systems involving mine action, child protection, and Women, Peace and Security (WPS). Its public-interest Informatics chapter examines the Muainishi application series, health-data governance, evidence about corporate conduct, and how to assess agricultural and carcinogenicity claims. This is contextual guidance, not clinical, legal, EOD, peacekeeper, or regulatory advice.

The prior edition explicitly left Linux kernel and shell-programming topics out of scope; this edition covers them in full learning chapters. Older Kali-tool directories remain a historical archive and are not a current package catalogue.

Operating-system choices

Raspberry Pi OS is Raspberry Pi's official OS and the default point of reference for standard Pi setup. Debian can be suitable for other deployments; hardware enablement and firmware support vary by image and board. Kali is a specialized security-testing distribution and is treated here as an optional, isolated lab environment, not as the default support OS.

These distributions share Linux foundations but have different repositories, package versions, defaults, and maintenance policies. Follow instructions for the OS actually installed. **Do not add Kali repositories to Debian or Raspberry Pi OS.**

Authoring environment and test scope

The authoring desktop is Debian GNU/Linux 13 (Trixie), ARM64, KDE Plasma 6.3.6 under Wayland. This is the environment used for local command and document checks. It does not imply that every Raspberry Pi model, official Raspberry Pi OS image, peripheral, Kali image, or African field deployment has been tested. Each page distinguishes general Linux advice, OS-specific guidance, source research, and behavior directly checked on the stated platform.

Zero-dependency reading and challenges

The guide is static Markdown and can be read on GitHub or locally without a site generator. The Base and Crypto challenge levels are designed to work from the book and standard command-line tools; no CTF server, added package, or external dataset is required. These are educational toy challenges, not flags for accessing live systems. Applications described elsewhere may have their own dependencies.

For current Raspberry Pi, Kali, and Debian facts, begin with the linked official documentation. For humanitarian and peace-support material, use the current mission mandate, national standards, and qualified specialists. Summaries here are learning aids, not substitutes for governing documents or role-specific training.

Guide Structure

The guide moves from practical support to foundational concepts, then to specialist contexts:

1. **Introduction and responsibility** — audience, scope, source standards, and working boundaries.
2. **Raspberry Pi field support** — choose an image, prepare a device, troubleshoot, protect user data, and hand over support clearly.
3. **Linux foundations and command-line recipes** — understand the kernel, Bash, and the effects of commonly used commands.
4. **Peace Operations field support** — support mine-action, child-protection, and WPS information flows without taking on specialist operational roles.
5. **Public-interest Informatics** — examine the Muainishi application series, health and agricultural data stewardship, corporate-accountability evidence, and how to evaluate cancer/GE-crop claims.
6. **Learn by doing** — solve Base, Quiz, and Crypto levels offline with no CTF server or extra package.
7. **CPIP- case study** — understand the Coffee Pot Internet Protocol and critically review its setup, dependencies, and security claims.
8. **Informatics portfolio** — explore selected work linked from William Majanja's two GitHub profiles, distinguishing forks, prototypes, and proprietary work.
9. **Historical Kali tool catalog** — browse surviving older material with the understanding that it is unreviewed, incomplete, and not current operating guidance.

Each maintained recipe follows a predictable format: task, platform and prerequisites, command, explanation, verification, safety/rollback, and sources. Examples use fictional data and local scope unless clearly labeled otherwise. Security exercises require systems the learner owns or is explicitly authorized to use.

The book is a static GitHub Markdown publication. It has no site generator or required build dependency; programs described in a chapter may have their own packages, hardware, or services.

Linux Topics Now Covered

The earlier edition explicitly left complex Linux fundamentals outside its scope. This guide now introduces the **Linux kernel**, **Bash and shell programming**, and the system concepts needed to understand the tools that run on Linux.

- Linux kernel and system model
- Bash shell programming
- Practical command-line recipes

These chapters focus on safe inspection and foundational learning. Advanced kernel changes belong in a disposable lab and must follow documentation for the target distribution and hardware.

The old text also asserted that linked tools and downloads were “completely verified” and “virus-free.” That assurance was not supportable and is withdrawn. A link is not a safety review; verify software provenance, signatures, hashes, maintenance status, and license before use.

William Majanja — Informatics Portfolio

Raspberry Pi for Technical Support Operatives in Africa is written and curated by William Majanja as an initiative of the **Hack with Raspberry Pi** community. William's public work across @WilliamMajanja and @WilliamMajanja-zz explores how information and software can support practical work in health, civic life, land, agriculture, publishing, education, and computing infrastructure.

The projects below are selected examples, not a claim that every repository on either account is original, complete, deployed, or independently validated. Forks, collaborations, research prototypes, and proprietary projects are identified. Descriptions summarize what the repositories say about themselves; they do not certify results or suitability for real-world decisions.

Selected work

Area	Project	Informatics focus and attribution
Bioinformatics	Muainishi Wa Jenomu	A browser-based genomics exploration/classification concept. Its README describes Gemini-backed analysis; because this can involve sending sequence data to an external service, treat privacy claims cautiously and never upload identifiable or sensitive genomic data without authorization and review. It is not a clinically validated diagnostic system.
Health and biomedical data	Muainishi Wa Viungo · Muainishi Wa Saratani	Repository-described concepts for transfusion/transplant information and AI-assisted oncology classification. These are research/learning prototypes, not validated clinical decision-support or diagnostic products.
Computational analysis	Partial-Realpart-Analysis- · CI-Lib	R/Bash-based statistical visualization and a computational-intelligence library spanning areas such as clustering, fuzzy logic, neural networks, and optimization. **CI-Lib is marked as a fork**; follow its upstream history and license before attributing all code to William.
Civic and election informatics	KuraYangu	Described by its repository as an election-observation and USSD voting system with results capture and parallel-vote-tabulation workflows. The project description is not independent evidence of election deployment, security, or readiness for official voting.
Geospatial and land information	atlaz	A land-intelligence and mapping project describing parcel analysis, source provenance, and scenario modelling. Its README marks the project **proprietary**; public visibility on GitHub does not make it open source.
Agricultural and edge informatics	AgriPulse-EdgeCluster · Minima-PiNet-OS	Project-described work on farm data, sensor telemetry, local inference, and Raspberry Pi edge systems. Hardware and deployment claims should be checked against current project documentation.

Area	Project	Informatics focus and attribution
Digital publishing and access	Babe's Bookstore	The site identifies William Majanja and presents itself as a public-domain digital library with topical browsing, online reading, and downloadable formats. Its catalogue and rights claims have not been independently audited by this guide. Its reader-oriented organization inspired this book's clear navigation and source/licensing notes; no text, books, or assets are copied.
Protocols, networks, and security learning	CPIP- · HydraCTF · StadiumMesh · Phantom	Work spanning the Coffee Pot Internet Protocol, CTF learning, mesh networking, and edge/media concepts. See the CPIP- guide and zero-dependency challenge track.

The Muainishi series

The three linked **Muainishi Wa Jenomu**, **Muainishi Wa Viungo**, and **Muainishi Wa Saratani** repositories form an author-associated health/bioinformatics application series. See the public-interest Informatics chapter for privacy, validation, and data-governance considerations. The repositories' clinical and model-performance claims have not been independently verified by this guide.

Peace Operations Training Institute

William's public GitHub profile states that he is on the **Peace Operations Training Institute (POTI) Honour Roll**. POTI describes the Honour Roll as listing students who completed courses and consented to have their names displayed. This guide records the profile's public statement; it does not claim specific course titles, completion dates, POTI employment, or course authorship.

- Peace Operations Training Institute
- POTI Honour Roll

At the time of review, `poti.org` resolved to an unrelated domain portfolio. The Institute's official public website is `peaceopstraining.org`.

Historical security artifact: Jester-themed ransomware fork

The older profile contains `fsociety-ransomware-MrRobot`, a C++ fork of `graniet/fsociety-ransomware-MrRobot`. Its README describes a Jester-themed ransomware proof of concept, warns that files may be unrecoverable, and calls the project incomplete. It is included only as a provenance-aware historical security artifact—not as a recommended tool or a project to run. No build or execution instructions are provided. GitHub reports no repository license, so do not assume the code is available for reuse.

Profiles and attribution

- WilliamMajanja — current profile and selected projects
- WilliamMajanja-zz — earlier profile and project archive

- ORCID record linked from the current profile

Project ownership, fork status, licenses, collaborators, and descriptions can change. Check the linked repository and its upstream project before reuse or republication. This portfolio does not relicense code, data, course material, or third-party assets.

Contributors and Historical Attribution

Original edition

The source edition of this book credits **Chandrapal** and the **Hack with Github** community. That historical credit remains in place for the material they contributed. The refreshed edition does not claim authorship of their original text.

Current edition

The new Linux foundations, command-line recipes, CPIP- guide, and Informatics portfolio are curated and written for this edition by **William Majanja** as an initiative of the **Hack with Raspberry Pi** community. See the author portfolio and the repository's licensing and provenance notes.

Contributions to future revisions are credited in the relevant pages or release notes. A GitHub username or fork relationship is not, by itself, proof of sole authorship of every file in a repository.

Acknowledgments

This edition is an initiative of the **Hack with Raspberry Pi** community. It acknowledges the original Hack with Github contributors named in the historical attribution page, and the maintainers of Kali Linux, Debian, GNU Bash, the Linux kernel, and the upstream projects linked throughout the guide.

The task-first recipe format was informed by Jason Cannon's *Command Line Kung Fu* (see References and Credits). The guide's examples and explanatory text are independently written; the source book is not reproduced here. The challenge track uses only the general idea of progressive flags from William Majanja's HydraCTF, without copying its code, database, or challenge material. The book's category-driven navigation also draws inspiration from William's Babe's Bookstore; its texts and assets are not reused.

Thanks to the maintainers of William Majanja's public projects for keeping their repositories available as references. Descriptions in this guide remain attributed to the project pages and do not imply endorsement or independent validation.

References and Credits

Raspberry Pi and Linux support

- Raspberry Pi: Getting started
- Raspberry Pi Imager and OS downloads
- Raspberry Pi configuration reference
- Raspberry Pi remote access
- Raspberry Pi operational security
- Debian documentation
- Linux kernel documentation
- GNU Bash Reference Manual

Kali Linux (optional lab platform)

- Kali Linux documentation
- Should I Use Kali Linux?
- Kali on ARM devices
- Updating Kali
- Kali Linux metapackages
- Kali network service policy
- Kali repository configuration
- Corey P. Schultz, *Kali Linux Cookbook*, Third Edition (2025), and its publisher sample-code repository. Further reading; confirm commands against current official Kali documentation.
- Kali Linux Revealed training

Recipe-format references

- Jason Cannon, *Command Line Kung Fu: Bash Scripting Tricks, Linux Shell Programming Tips, and Bash One-liners*, CreateSpace, 2014, ISBN 978-1-4992-2203-6. A local copy informed the task-grouped recipe format. This guide does not reproduce its text or examples.
- Babe's Bookstore, by William Majanja. Its category-based browsing and rights-forward presentation informed this guide's navigation; no wording, books, or assets are copied.

Peace operations, humanitarian protection, and standards

- POTI/UNMAS, Mine Action and Explosive Hazard Management
- POTI, Child Protection for UN Military
- UN Women/POTI, Women, Peace and Security in Africa
- International Mine Action Standards
- IMAS 08.10: Non-technical survey
- IMAS 13.10: Victim assistance
- UNMAS: Victim assistance in mine action
- UN Security Council resolutions archive
- UN Women: WPS global norms and standards
- UN Office of the Special Representative for Children and Armed Conflict
- POTI official site and Honour Roll

Protocol and learning-project references

- RFC 2324: Hyper Text Coffee Pot Control Protocol
- RFC 7168: HTCPCP-TEA
- CPIP- repository
- HydraCTF repository, cited as high-level inspiration for progressive flags only; no code or challenge text is reproduced.

Public health, agriculture, and corporate accountability

- WHO: Food, genetically modified — product-specific, case-by-case food-safety assessment guidance.
- National Academies: Genetically Engineered Crops—Experiences and Prospects
- IARC: Glyphosate evaluation and hazard vs. risk
- WHO: Tobacco and nicotine
- US DOJ: United States v. Purdue Pharma L.P.
- US FTC: Facebook privacy penalty and restrictions
- US FTC: Cambridge Analytica matter

Attribution and reuse

These references are for further reading. Their authors retain their own copyrights and licenses. A link or citation does not grant permission to republish a book, course, repository, image, or code sample. See Licensing and Provenance.

Raspberry Pi Field Support

This part of the book is the practical core for technical support operatives maintaining Raspberry Pi systems. It offers repeatable first-line checks, recovery planning, and safe handover practices for offices, schools, community networks, labs, NGOs, and other deployments. Actual needs differ by site; assess local power, connectivity, language, accessibility, hardware supply, and organizational policy instead of assuming one African operating environment.

Field-support path

1. Identify hardware and choose an operating system.
2. Use a consistent triage process.
3. Plan backups, updates, handover, and sustained support.
4. Learn the kernel, Bash, and practical command line.

Keep the platform identity clear

- **Raspberry Pi OS** is the official operating system published by Raspberry Pi; its Imager and support documentation are the primary reference for standard Pi deployments.
- **Debian** is a separate general-purpose distribution. Raspberry Pi hardware support depends on the specific board, image, kernel, and firmware.
- **Kali Linux** is a specialized security-testing distribution. It is an optional, authorized lab platform in this book, not the default support image for day-to-day Pi deployments. Check Kali's current ARM device support before selecting a Kali image.

Use the OS-specific documentation and repositories for the OS installed on the device. Never mix Kali repositories into Debian or Raspberry Pi OS to "get a tool."

Test scope

The guide authoring environment is Debian 13 (Trixie), ARM64, KDE Plasma 6.3.6 on Wayland. This confirms the documented authoring environment only. It does **not** mean every Pi model, Raspberry Pi OS image, peripheral, or Kali ARM image has been tested. Recipes identify their assumptions and direct readers to current vendor documentation for hardware-dependent details.

Primary references

- Raspberry Pi: Getting started
- Raspberry Pi OS downloads and Raspberry Pi Imager
- Raspberry Pi: Configuration
- Raspberry Pi: Remote access
- Raspberry Pi: Operational security

Imaging and First Boot

1. Identify the deployment before writing an image

Record the board model and revision, intended role, architecture, peripherals, network connection, storage type, and recovery plan. Check the official compatibility and power requirements for that exact model. Raspberry Pi models differ in boot media, power connectors, wireless bands, firmware, and peripheral support; do not infer one model's requirements from another.

For support work, decide in advance:

- Is this a desktop, headless server, kiosk, classroom device, gateway, or appliance?
- Who owns the device and who is authorized to administer it?
- Which data will it hold, how sensitive is that data, and how will it be backed up?
- What network is it allowed to join, and is remote administration required?
- Can the device be recovered locally if the network or remote service is unavailable?

2. Choose the operating system

Use the operating system supported for the intended workload and hardware. For a standard Raspberry Pi deployment, begin with Raspberry Pi's official OS options. A Debian image may be appropriate for some support workloads, but hardware enablement can differ from Raspberry Pi OS. Kali ARM images are for authorized security testing and should be kept separate from routine support deployments.

Check the target image's release date, update policy, architecture, hardware support, and security maintenance before deployment. Store the image source, version, checksum or signature information where available, and date in the device's support record.

3. Prepare boot media carefully

Raspberry Pi Imager can write supported operating-system images to microSD and other supported storage. **Imaging erases the selected destination.** Before writing:

1. Make sure the selected storage device is the intended card or drive.
2. Disconnect unrelated removable storage where practical.
3. Keep Imager's protection against selecting the computer's system disk enabled.
4. Use the official image source and complete the verification step offered by the tool.
5. Never use a production or evidence disk as test media.

Imager can also configure a hostname, user account, locale, wireless network, and remote access before first boot. Choose unique credentials; do not use a shared password across a fleet, and never include credentials in notes, screenshots, scripts, or this guide.

4. First boot checklist

For a desktop system, connect only the required display, keyboard, pointing device, network, and correct model-specific power supply. For a headless system, preconfigure network and an approved remote-access method before writing the card, then confirm the device identity through a trusted channel before logging in.

At first boot:

- Set the correct hostname, locale, timezone, keyboard layout, and administrative user.
- Confirm the intended network and verify that the device receives the expected address.
- Enable SSH or another remote-support feature only when needed; keep its access limited and authenticated.
- Apply OS updates according to the installed distribution's official guidance and an agreed maintenance window.
- Record OS version, kernel release, architecture, device model, storage, and enabled services.
- Confirm a clean shutdown and a recoverable backup before handing the device over.

On Raspberry Pi OS, use its supported configuration interfaces. Tools such as `raspi-config` are not guaranteed to exist or behave the same way on generic Debian or Kali images.

5. Headless network setup

Ethernet is a useful first diagnostic where available. If Wi-Fi is required, confirm the network name, credentials, security mode, signal at the device location, and the board's supported band. Some older Pi boards support 2.4 GHz only; check the model specification rather than guessing.

Use approved network settings. Do not connect a support device to an untrusted access point merely to "see if it works," and do not scan other clients on the network as a connectivity test. If preconfigured Wi-Fi fails, use a local console or wired connection and follow the official Raspberry Pi OS documentation for the installed release. The old `wpa_supplicant.conf` boot-folder method is not applicable to current Raspberry Pi OS releases.

6. Power, shutdown, and hardware safety

Use a supply and cable suitable for the exact Pi model and its attached peripherals. Undersupply, damaged cables, poor connectors, and high peripheral load can cause instability or storage corruption. Check the official power supply guidance before diagnosing a board as faulty.

Use the operating system's shutdown function or `sudo poweroff` before disconnecting power. A hard power cut can corrupt filesystems or lose data. Disconnect power before attaching or removing HATs, GPIO wiring, storage, or other hardware. GPIO pins are low-voltage logic; never connect them directly to mains or unknown voltages. Follow the board and accessory manufacturer's electrical safety guidance.

7. References

- Official Raspberry Pi getting-started guide
- Raspberry Pi Imager
- Raspberry Pi boot diagnostics
- Raspberry Pi GPIO documentation

Support and Troubleshooting Playbooks

Use the same sequence for every incident: **confirm authorization and impact → observe → record → form a hypothesis → make one reversible change → verify → document or escalate**. These playbooks are first-line support checks, not substitutes for board-specific diagnostics or specialist repair.

1. Capture a safe baseline

Run locally or over an approved SSH session:

```
date --iso-8601=seconds
hostnamectl
uname -a
cat /etc/os-release
df -hT
free -h
uptime
ip -brief address
ip route
systemctl --failed --no-pager
```

These are primarily inspection commands. Some may be absent on minimal images; record that rather than installing packages during initial triage. Do not publish raw output without reviewing it for usernames, hostnames, addresses, serial numbers, and sensitive operational details.

2. Device does not boot

1. Confirm the model and power supply against the manufacturer's requirements; check the cable and connector.
2. Disconnect nonessential peripherals, then retry with known-good, compatible power and display equipment.
3. Allow the first boot time to complete. If it still does not start, read the board's status LEDs using the **model-specific** warning-code table.
4. Check that the correct OS image was written to the intended storage and that the image verification step passed.
5. If re-imaging is necessary, first confirm that all needed data has a separate backup. Re-imaging is destructive.
6. If the board still fails, preserve the old media and logs where possible, and escalate to hardware or OS support.

Do not repeatedly remove power from a device that may still be writing to storage. Raspberry Pi's current troubleshooting guidance and boot diagnostic information are authoritative for model-specific indicators.

3. Device boots but has no network

Start locally and inspect links and routes:

```
ip -brief link
ip -brief address
ip route
```

Then check the physical connection, Wi-Fi band support, signal, approved SSID, credentials, DHCP availability, and whether the network requires a captive portal or device registration. If NetworkManager is installed, `nmcli device status` can help inspect device state; its absence does not prove a fault.

Test in layers, using only the organization's approved gateway and resolver. A successful local address does not prove DNS, and successful DNS does not prove an external service is healthy. Do not change firewall rules, disable authentication, or scan neighboring devices to troubleshoot one Pi.

4. Remote access is unavailable

- Verify the expected hostname/address through the authorized device inventory or local console.
- Confirm that SSH or the approved remote-support service is installed, enabled, and bound only as intended.
- Check host-key identity before entering credentials. A changed SSH host key is a warning to investigate, not a prompt to bypass verification.
- Inspect service status and recent logs with `systemctl status --no-pager SERVICE` and `journalctl -u SERVICE --since today --no-pager`.
- Confirm that network policy, VPN, firewall, and NAT rules permit the approved path.
- Revoke credentials or device access through the designated administrator if a device is lost or compromised.

Never expose SSH or a desktop-sharing server directly to the public internet as a quick fix. Prefer the organization's approved VPN or managed remote-support system, with MFA where available.

5. Desktop or display problem

Check monitor input, cable, power, resolution support, and whether the problem exists before the desktop starts. Record display model and connection type. In KDE/Wayland sessions, screen sharing and capture use desktop portal permissions; an X11-specific recipe or automation script may not behave the same under Wayland.

Inspect the current session without changing it:

```
printf 'Session: %s\n' "${XDG_SESSION_TYPE:-unknown}"
printf 'Desktop: %s\n' "${XDG_CURRENT_DESKTOP:-unknown}"
```

These environment variables are informative but not a complete display diagnostic. Keep SSH or a local TTY recovery route available before changing display-manager, compositor, or graphics-driver settings.

6. Slow, hot, or unstable device

Start with read-only resource checks:

```
uptime
free -h
df -hT
ps -eo pid,comm,%cpu,%mem --sort=-%cpu | head
journalctl -k -p warning --since today --no-pager
```

Use these as clues, not diagnoses. Check ventilation, ambient conditions, power supply, attached USB devices, storage health, and recent package or configuration changes. Follow model-specific official guidance for thermal indicators and do not remove heatsinks or alter clock/voltage settings without understanding warranty and reliability implications.

7. Service failure

For a known service name:

```
systemctl status --no-pager --full SERVICE
journalctl -u SERVICE --since today --no-pager
```

Capture the exit code, error message, relevant recent change, and exact service name. Do not use `systemctl enable`, `disable`, `restart`, or root privileges until the service's role and change authorization are clear. Kali intentionally has a different network-service default policy from a typical desktop distribution; consult Kali's policy before changing one.

8. Storage or update problem

First inspect free space with `df -hT` and identify large directories within data you are authorized to inspect. Do not delete logs, package databases, or user files simply to free space. Preserve logs and note package-manager errors before attempting recovery.

Use the update process for the installed OS. Raspberry Pi OS, Debian, and Kali can share APT concepts, but package sources, firmware, service policies, and release models differ. For Kali rolling, schedule updates outside a live assessment and read Kali's current update guidance. Never add Kali sources to Debian or Raspberry Pi OS.

9. Escalation notes

When escalating, include the device model, OS/image version, kernel and architecture, symptom and first occurrence, recent changes, relevant sanitized output, reproduction steps, expected vs observed behavior, steps attempted, backup state, and impact. Do not attach private keys, passwords, raw child-protection records, casualty data, or unredacted mine-hazard coordinates to a public ticket.

Field Operations, Maintenance, and Handover

1. Build a supportable deployment

Before a Pi enters service, record who owns it, what it does, who may administer it, what data it handles, how it connects, and how it can be recovered. Use a unique hostname and device identity; keep the inventory private when it contains site or operational information.

Design for the conditions at the actual deployment site. Where power interruptions, costly connectivity, long travel to a repair location, multiple working languages, or limited spare parts are relevant, plan around those constraints. Do not assume those conditions apply everywhere in Africa or to every organization.

Practical resilience measures include:

- a compatible power supply and known-good cable for the board and peripherals;
- a labeled, verified recovery image and spare boot media where appropriate;
- a tested backup and restore procedure for configuration and user data;
- a documented offline troubleshooting path and local copy of essential manuals;
- clear ownership for updates, credentials, incident reporting, and replacement;
- an escalation contact and handover record in the users' preferred language where feasible.

2. Updates and change control

Do not update a production or mission device opportunistically. Record the current state, read release notes, back up necessary data, agree a maintenance window, and know how to recover. Change one class of component at a time and verify the service after reboot.

Use only the operating system's supported repositories. Raspberry Pi OS firmware and tooling, Debian stable packages, and Kali rolling packages do not have identical lifecycles. Follow the applicable upstream instructions rather than reusing a command simply because the distributions share APT.

For an authorized Kali lab, the current official guidance describes `apt update` followed by `apt full-upgrade`; review proposed package changes and schedule outside an engagement. For Raspberry Pi OS and Debian, check each distribution's release-specific instructions before applying updates. Do not use `-y` on an unfamiliar or operational system merely to make an example shorter.

3. Backups and recovery

A backup is useful only if it can be restored. Define:

- which user data, service configuration, and application state must be retained;
- where copies are stored and who may access them;
- how encryption keys and recovery credentials are protected;
- how often backups run and how long they are retained;
- how a restore is tested without overwriting the live deployment.

Keep at least one backup separate from the device and verify it periodically. Record the exact image and package state needed to rebuild. A whole-disk image contains deleted space and sensitive data; store it as confidential material. Re-imaging an SD card or SSD overwrites its contents, so confirm the target and backup before writing.

4. Data protection and localization

Collect only the support data needed to resolve an incident. Minimize personally identifying information, avoid publishing network maps or exact sensitive site coordinates, and follow applicable organizational policy and local law. Protect service logs and backups; access should be role-based and time-limited.

Make support materials usable by the people who will rely on them: use clear language, explain acronyms, provide a compact checklist for low-connectivity situations, and account for accessibility needs. Translation should be reviewed by a fluent local speaker rather than assumed from machine output.

5. Handover template

```
Device / hostname:
Model and storage:
Operating system / image date:
Kernel / architecture:
Purpose and owner:
Network and approved remote-access method:
Backup and recovery location:
Known issues / peripherals:
Last maintenance date:
Responsible support contact:
Next review date:
```

Do not put passwords, private keys, access tokens, or sensitive beneficiary information in this handover. Store such material in the organization’s approved credential or case-management system.

6. Hardware handling

Shut down before moving storage or connecting a HAT. Disconnect power before changing GPIO wiring. Use GPIO only with compatible low-voltage hardware and reviewed pinout documentation; Raspberry Pi GPIO is not a mains interface. Have qualified electrical personnel design or inspect mains-powered equipment and enclosures.

When a unit returns from the field, record condition, preserve evidence if an incident is suspected, check storage and power before reimaging, and wipe or retire data according to the organization’s policy.

7. Authoritative references

- Raspberry Pi security guidance
- Raspberry Pi remote access
- Raspberry Pi OS updates
- Kali update policy
- Debian documentation

Linux Foundations

These chapters provide the operating-system and shell knowledge needed to support Raspberry Pi and Linux systems thoughtfully. They apply to Raspberry Pi OS, Debian, and Kali where noted; distribution-specific commands are labeled.

Chapters

- Linux kernel and system model
- Bash shell programming
- Practical command-line recipes

Platform tested for this edition

The authoring desktop is Debian GNU/Linux 13 (Trixie), ARM64, KDE Plasma 6.3.6 under Wayland. Commands described as *verified on the authoring system* are limited to that environment. A Debian result does not establish that a Kali-only package or workflow works on Kali, and vice versa.

Kali is a Debian-derived distribution with its own repositories, packaging, defaults, and release process. **Do not add Kali repositories to a Debian installation** to obtain a Kali package. Use a supported Kali virtual machine, dedicated device, or isolated lab when a lesson specifically requires Kali.

The book itself is plain GitHub-flavored Markdown. No book builder or third-party documentation package is required to read it on GitHub.

Linux Kernel and System Model

This chapter introduces the Linux kernel as the privileged core of the operating system. It focuses on understanding and safely inspecting a running system; kernel replacement, module loading, and tuning are advanced administration tasks and should be practiced in a disposable virtual machine.

1. What the kernel does

The **kernel** mediates access to hardware and provides services to programs. It manages processes, memory, devices, filesystems, networking, permissions, and system calls. A Linux distribution combines a Linux kernel with user-space programs, libraries, an installer, a package repository, and project-specific defaults.

The kernel is not the same thing as a distribution, desktop, shell, or collection of security tools:

Component	Responsibility	Example
Kernel	Hardware and core operating-system services	Linux kernel, device drivers
User space	Programs and libraries that use kernel services	`systemd`, OpenSSH, Python
Distribution	Integrated system, packages, repositories, defaults	Debian, Kali Linux
Desktop environment	Graphical session and desktop applications	KDE Plasma
Shell	Command interpreter and scripting language	Bash, Dash, Zsh

Programs request kernel services through **system calls**. For example, opening a file in a program ultimately asks the kernel to resolve a path, check permissions, and interact with the filesystem. Ordinary applications run in user space; kernel code runs with elevated privilege. A kernel vulnerability can therefore affect the whole machine, while a user-space program is generally constrained by its account and permissions.

2. Boot and login at a high level

A typical Linux boot sequence is:

1. Firmware initializes hardware and selects a boot device.
2. A bootloader selects a kernel and initial RAM filesystem (initramfs).
3. The kernel initializes memory, drivers, and devices, then starts the first user-space process.
4. The init system starts system services.
5. A display manager may start a graphical session such as KDE Plasma.
6. A login manager or terminal starts the user’s applications and shell.

Exact details vary by hardware and distribution. On ARM single-board computers, firmware and boot partitions can differ from a conventional UEFI PC. A Wayland session is a user-space graphical session; Wayland does not replace the Linux kernel.

3. Identify the running system

These inspection commands do not change system configuration:

```
uname -r                # running kernel release
uname -m                # machine architecture, e.g. aarch64 or x86_64
cat /etc/os-release     # distribution identity and release
ps -p 1 -o comm=        # name of the first user-space process
```

`uname -r` reports the **currently running** kernel. Installing a newer kernel package does not make it active until the system has booted into it. `/etc/os-release` identifies the distribution; it does not identify every repository or package source configured on a system.

On this guide's authoring machine, `dpkg --print-architecture` reports `arm64`, `/etc/os-release` reports Debian 13 (Trixie), and the graphical session is Wayland with KDE Plasma. These values are a test baseline, not a claim about every reader's machine.

4. Processes, memory, devices, and virtual filesystems

The kernel tracks processes and their resources. Each process has an identity, credentials, open files, memory mappings, and scheduling state. Linux exposes selected runtime information through virtual filesystems:

- `/proc` exposes process and kernel runtime information.
- `/sys` exposes device and kernel-object attributes.
- `/dev` contains device nodes used to access devices through defined interfaces.

These are not ordinary directories containing conventional files on disk. Some entries are writable controls. Treat unfamiliar `/proc` and `/sys` paths as interfaces, not as harmless text files; prefer reading official documentation before changing a value.

Useful read-only inspection examples:

```
cat /proc/cmdline       # kernel command-line options for this boot
cat /proc/meminfo       # memory counters
ls /sys/class           # device-class names exposed by sysfs
lsmod                   # currently loaded modules
```

`lsmod` lists kernel modules; it does not prove that a module is safe or needed. `modinfo NAME` displays metadata where available. Do not load or unload modules on a production system as an experiment.

5. Drivers and modules

Many drivers are built into the kernel; others are loadable **kernel modules**. Modules can add support for hardware or kernel features. Because a module executes with kernel privilege, module provenance and compatibility matter. Secure Boot policies, signing requirements, kernel configuration, and distribution packaging can restrict which modules load.

For inventory and diagnostics, use the distribution's documentation and package database. Do not download a random `.ko` file or copy one from a different kernel release. Modules are tightly coupled to kernel versions, configuration, and architecture.

6. Kernel updates and distribution differences

Use the package manager and repositories for the distribution actually installed:

- **Debian:** follow Debian's documentation and configured Debian repositories.
- **Kali:** follow Kali's documentation and Kali's supported update process.

Never mix Kali and Debian repositories on one installation as a shortcut. That can replace foundational packages and leave the system in an unsupported state. Before a kernel update, make a backup, read the relevant distribution advisory, ensure there is a recovery route, and schedule a reboot when appropriate.

Kernel headers are development files used by some out-of-tree modules and build systems. They are not the running kernel, and installing headers alone does not update or activate a kernel.

7. Kernel configuration and tuning

Kernel configuration includes boot parameters, module settings, sysctl values, and distribution defaults. Some settings affect networking, memory, security boundaries, or device behavior. The fact that a value is writable does not mean it should be changed.

For a safe workflow:

1. Record the current OS, architecture, kernel release, and the setting's current value.
2. Find the distribution or upstream kernel documentation for that exact setting.
3. Test in a disposable VM or non-production device with a recovery path.
4. Change one item at a time and verify the expected behavior.
5. Record how to revert the change and whether it persists across reboot.

Avoid copying "kernel hardening" snippets without understanding their scope. A setting may be obsolete, unavailable in a particular kernel, or harmful to a desktop or hardware driver.

8. Building a kernel: when and where

Building a custom kernel involves selecting a source version, validating its origin, configuring options, compiling for the correct architecture, signing where required, packaging, installing, and maintaining a recovery kernel. A custom kernel is not automatically faster or more secure. It can lose vendor patches, hardware support, security updates, or bootability.

This guide does not instruct readers to replace the kernel on their daily workstation. If kernel development is a learning goal, use a disposable virtual machine or spare device, follow the distribution's maintained packaging instructions, preserve a known-good kernel, and verify checksums and signatures from trusted release channels.

9. Security concepts

- **Least privilege:** run ordinary programs as an unprivileged user; elevate only a specific administrative action.
- **Namespaces and cgroups:** kernel primitives used by containers to isolate views and limit resources. A container still shares the host kernel.
- **Capabilities:** divide some root privileges into narrower permissions; they are not a substitute for understanding the program being granted them.

- **Kernel attack surface:** includes system calls, drivers, modules, and exposed device interfaces. Keeping the supported system updated and removing unneeded access are more reliable than blindly applying tuning snippets.
- **Logs and evidence:** kernel messages can help diagnose drivers and boot problems. Preserve timestamps and original logs during incident response.

10. References

- The Linux Kernel documentation
- Debian documentation
- Kali Linux documentation
- Linux man-pages project

Scope: foundational education. Commands above are read-only examples; kernel modification procedures should be validated separately on the target distribution and hardware.

Bash Shell Programming

Bash is both an interactive shell and a scripting language. Learning shell fundamentals makes Linux administration more reproducible and helps readers understand the command examples elsewhere in this guide. The examples use Bash features and do not require extra packages beyond Bash and standard command-line utilities already present on many desktop installations.

1. Commands, arguments, and exit status

A simple command consists of a program name followed by arguments:

```
printf '%s\n' 'Hello, Linux'
```

The shell locates the command through `PATH`, expands syntax such as variables and globs, and then starts the program. The program returns an **exit status**: conventionally `0` means success and a non-zero value means some kind of failure. Capture the previous status with `$?` immediately after the command that produced it.

```
if test -r /etc/os-release; then
    printf '%s\n' 'Distribution metadata is readable.'
fi
```

Do not treat “the command printed output” as proof that it succeeded. Check exit status and inspect expected output.

2. Quoting and expansion

Quoting controls what the shell interprets:

- **Single quotes** preserve every character literally: `'two words $HOME'`.
- **Double quotes** preserve spaces and most special characters while allowing variable and command substitutions: `"$HOME"`.
- **Unquoted expansions** are subject to word splitting and pathname expansion; avoid them for data.

Prefer:

```
printf '%s\n' "$HOME"
```

Avoid relying on unquoted variables, especially when a value may contain spaces, wildcard characters, or be empty:

```
# Safer: one argument, even if the path contains spaces.
mkdir -- "$HOME/security notes"
```

The `--` convention marks the end of options for many utilities. It helps when a filename begins with `-`, but it is not supported by every command.

3. Variables and environment

Assign a shell variable without spaces around =:

```
project='sample-lab'
printf 'Project: %s\n' "$project"
```

An **environment variable** is exported to child processes:

```
export LANG=C
```

Environment values are configuration, not secure secret storage. Avoid placing passwords, private keys, or long-lived tokens directly in shell history, command arguments, shared logs, or public scripts.

4. Redirection and pipelines

File descriptors conventionally include standard input (`0`), standard output (`1`), and standard error (`2`).

```
command >output.txt 2>errors.txt
command >>output.txt
command 2>&1 | another-command
```

> truncates an existing destination. Check the path before using it; a typo can overwrite valuable data. A pipeline connects one process's output to another process's input:

```
printf '%s\n' alpha beta gamma | sort
```

By default, a pipeline's status is generally the status of its last command. In scripts, `set -o pipefail` makes a pipeline fail if a command in it fails.

5. Globs, arrays, and loops

Globs are expanded by the shell, not by the command. An unmatched glob may remain a literal string in Bash unless `nullglob` is enabled.

```
shopt -s nullglob
files=(./*.md)
for file in "${files[@]}"; do
    printf 'Markdown file: %s\n' "$file"
done
```

Arrays preserve each item as a separate value. Quote `"${array[@]}"` when passing elements as arguments. This avoids turning a filename containing spaces into multiple arguments.

6. Conditions and functions

Use `[[ ... ]]` for Bash string tests and `(( ... ))` for arithmetic:

```
if [[ -n ${EDITOR:-} ]]; then
    printf 'Editor is set to %s\n' "$EDITOR"
fi

count=3
if (( count > 0 )); then
    printf 'There are items to review.\n'
fi
```

`${value:-fallback}` supplies a default without changing the variable. Define a function to give a repeated operation a name:

```
say_status() {
    printf '[%s] %s\n' "$(date '+%H:%M:%S')" "$1"
}

say_status 'Review started'
```

7. A small, safe script

This example only reads Markdown files in the current directory and reports how many it finds:

```
#!/usr/bin/env bash
set -Eeuo pipefail
shopt -s nullglob

files=(./*.md)
printf 'Found %d Markdown files in %s\n' "${#files[@]}" "$PWD"
for file in "${files[@]}"; do
    printf ' - %s\n' "$file"
done
```

`set -e` can behave unexpectedly in conditionals, pipelines, and functions; it is not a substitute for deliberate error handling. For scripts that modify data, validate arguments, quote every expansion, test on sample data, and make backups or use a disposable working directory.

Syntax-check a script without running it:

```
bash -n ./script.sh
```

Run a local script explicitly with Bash:

```
bash ./script.sh
```

8. Safe scripting practices

1. **Be explicit about paths.** Resolve input and output paths before destructive actions.
2. **Quote expansions.** Treat filenames and user input as data, not shell syntax.
3. **Avoid `eval`.** It turns text into executable shell code and is rarely necessary.
4. **Avoid broad deletion and privilege.** Do not run `rm -rf` on variables or use `sudo` as a debugging shortcut.
5. **Check prerequisites and errors.** A clear error is safer than continuing with incomplete state.
6. **Do not parse `ls` output.** Use globs or `find` with null-delimited output when appropriate.
7. **Keep secrets out of source.** Use a suitable secret store or a protected file with carefully limited permissions.
8. **Test in a disposable directory.** First test with harmless sample files and inspect the result.
9. **Know which shell is used.** Debian's `/bin/sh` is commonly Dash; Bash-specific syntax requires Bash (`#!/usr/bin/env bash`).

9. Bash versus other shells

Shells share common concepts but do not share every feature. Bash arrays, `[[ ... ]]`, process substitution, and some expansions are not POSIX `sh` syntax. If portability is required, write to the POSIX shell subset and test with the intended `/bin/sh`. If a script uses Bash, declare Bash in its shebang and invoke it accordingly.

10. References

- Practical command-line recipes in this guide
- GNU Bash Reference Manual
- BashFAQ
- ShellCheck — optional static-analysis service/tool; not required to read or run the examples in this guide.
- `bash(1)` manual

Practical Command-Line Recipes

This chapter uses a cookbook format: each recipe starts with a practical task, gives a short command, then explains what it does, how to check the result, and what to watch out for. Commands are original examples for this guide, not copied from a third-party book. The default examples favor inspection and reversible work over destructive actions.

Compatibility key

- **Debian/Kali:** expected to work on current Debian-derived systems; check optional command availability.
- **Bash:** requires Bash syntax; Debian's `/bin/sh` may be Dash.
- **Optional:** requires the named package or service, which may not be installed by default.

Quick recipe index

Topic	Recipes
Identify the device and safely reuse shell history	1, 2
Format, count, and process local text	3, 4, 5
Inspect storage, sockets, services, and routes	6, 7, 8, 9
Check packages, update Kali, and compare remote files	10, 11, 12
Verify, preview, serve, or record local files	13, 14, 15, 16
Everyday habits	17

1. Identify the distribution, architecture, and kernel

Use when: a command or package depends on which Linux system is running.

```
grep -E '^(PRETTY_NAME|VERSION_CODENAME)=' /etc/os-release
uname -m
uname -r
```

How it works: `/etc/os-release` reports distribution identity; `uname -m` reports machine architecture; `uname -r` reports the currently running kernel. These are distinct facts: the installed packages, CPU architecture, and active kernel release are not interchangeable.

Verify: compare the distribution and release with the official documentation for the command you plan to use.

Platform note: on Debian/Kali, `dpkg --print-architecture` gives the package architecture (for example, `arm64`), which may differ in spelling from `uname -m` (often `aarch64`).

2. Recall a shell command without blindly rerunning it

Use when: you want to reuse a long command from the current shell session.

Press **Ctrl-R**, type a distinctive fragment, inspect the recalled command, edit it, and press Enter only after checking every argument and target.

Why this safer pattern: Bash history expansion offers shortcuts such as `!!`, `!$`, and `!prefix`, but they can repeat a destructive command or substitute an unexpected argument. In particular, avoid blindly prefixing a recalled command with `sudo`. Read the complete command first and add elevation deliberately.

Privacy note: command history may contain paths, hostnames, tokens, or other sensitive data. Clearing the visible history is not a secure erasure guarantee; commands may also be present in terminal logs, audit records, backups, or process accounting.

3. Format tab-separated output as a table

Use when: you have a small tab-separated report that is hard to scan.

```
printf 'name\tstate\nrouter\tseen\nserver\tunknown\n' | column -t -s $'\t'
```

How it works: `printf` emits predictable tab-separated input. `column` aligns fields using a tab as the delimiter.

Verify: compare the output rows with the input. Do not use this presentation-oriented output as a machine-readable data format.

Platform note: `$'…'` is Bash syntax. Some minimal systems may not include `column`; it is commonly provided by `util-linux` on Debian-derived systems.

4. Count repeated lines

Use when: summarizing identical lines from a local, authorized log or report.

```
printf '%s\n' warning info warning error warning | sort | uniq -c | sort -nr
```

How it works: `uniq` counts adjacent duplicates, so sort first. The final numeric reverse sort puts the most common values first.

Caution: raw logs may contain personal data, internal addresses, session identifiers, or secrets. Redact before sharing output.

5. Read a file one line at a time in Bash

Use when: writing a small Bash script that must preserve spaces and backslashes in each line.

```
while IFS= read -r line || [[ -n $line ]]; do
    printf 'Line: %s\n' "$line"
done < ./sample.txt
```

How it works: `IFS=` prevents trimming; `-r` prevents backslash interpretation; the final condition handles a last line without a newline. Redirecting the file into the loop avoids the common pipeline subshell issue.

Verify: test with a disposable file containing spaces, a backslash, an empty line, and a final line without a newline. Use only files you are authorized to read.

6. Inspect disk usage

Use when: checking which mounted filesystems are nearing capacity.

```
df -hT
```

How it works: `df` reports filesystem-level use; `-h` selects human-readable units and `-T` includes the filesystem type.

For a user-owned directory, a limited-depth summary can help find large subdirectories:

```
du -h --max-depth=1 "$HOME" 2>/dev/null | sort -h
```

`du` may take time on large trees. Start with your own home directory; do not recursively scan private or production data without permission.

7. Check local listening sockets

Use when: confirming whether a service is listening on this machine.

```
ss -lntup
```

How it works: `ss` displays listening TCP and UDP sockets; `-n` keeps numeric addresses/ports, `-t` selects TCP, `-u` UDP, `-l` listeners, and `-p` process information where available.

Verify: check the local address as well as the port. `127.0.0.1` is loopback; `0.0.0.0` and `::` generally indicate listening on all IPv4 or IPv6 interfaces. Process details for other users' services may require elevated privileges; do not use `sudo` unless you need it and are authorized.

8. Read service status and logs

Use when: diagnosing a systemd service on Debian or Kali.

```
systemctl status --no-pager --full NetworkManager
journalctl -u NetworkManager --since today --no-pager
```

Replace `NetworkManager` with a service you recognize. `status` shows current state and recent messages; `journalctl` queries the journal.

Caution: logs can contain usernames, network names, addresses, and service data. Sanitize them before posting publicly. These commands inspect state; `start`, `stop`, `enable`, and `disable` change it.

9. Inspect network configuration without scanning

Use when: checking local interfaces and the current route.

```
ip -brief address
ip route
```

These commands inspect local configuration. They do not test whether another person's system is vulnerable or authorized for assessment. If DNS resolution is needed, use an approved hostname and understand that the lookup contacts the configured resolver.

10. Check a package before installing it

Use when: verifying which version a distribution repository offers.

```
apt-cache policy PACKAGE_NAME
```

Replace `PACKAGE_NAME` with an actual package name. `apt-cache policy` is an inspection step; it does not install the package.

Kali-specific reminder: follow Kali's repository guidance on a Kali installation. On Debian, use Debian repositories. Do not add Kali repositories to Debian or mix Debian and Kali repositories on one installation.

11. Update Kali deliberately

Use when: maintaining a Kali installation between authorized engagements.

Kali is rolling-release. First review Kali's current update guidance and confirm the configured repository. On a Kali installation, the documented upgrade flow is:

```
sudo apt update
sudo apt full-upgrade
```

Review proposed removals and changes before confirming. Back up important data, make sure you have recovery access, and avoid changing a known-good assessment environment during an engagement. These commands are for Kali, **not** a Debian host. Debian has its own release and upgrade procedures.

12. Compare a local file with an authorized remote copy

Use when: checking whether a local file differs from the same file on a host you administer.

```
diff -u ./local.conf <(ssh admin@example.test 'cat -- /etc/example.conf')
```

How it works: Bash process substitution makes the SSH command's output available to `diff`. Replace the account, host, and path with values you are authorized to access. Review the SSH host-key prompt; do not bypass host-key verification.

Caution: configuration output may contain secrets. Avoid writing it to public logs or tickets.

13. Verify a downloaded file against its publisher's checksum

Use when: a trusted publisher provides a checksum for the exact release artifact.

```
sha256sum ./downloaded-file
```

Compare the full digest with the value obtained independently from the publisher's authenticated release channel. Computing a hash alone does not establish authenticity; if the download and checksum come from the same compromised source, both may be wrong.

14. Preview an archive before extracting it

Use when: examining a downloaded archive before use.

```
file ./archive.tar.gz
tar -tzf ./archive.tar.gz | less
```

Review member names and sizes before extraction. Archives from untrusted sources can contain misleading paths, symlinks, or unexpected content. Extract only in a new, disposable directory and inspect the result before opening files or running programs.

15. Serve a local directory only to this computer

Use when: previewing static files in a browser without installing a web server.

From the directory you intend to share:

```
python3 -m http.server --bind 127.0.0.1 8000
```

Open `http://127.0.0.1:8000/` on the same machine, then stop the server with `Ctrl-C`.

Why bind matters: Python's simple HTTP server can otherwise listen on network interfaces. Do not use this command to serve confidential files or expose a directory to a network. It is a convenience server, not an authenticated production server.

16. Capture a terminal session carefully

Use when: keeping a transcript of a reproducible local test.

```
umask 077
script --quiet --return "$HOME/session.typescript"
```

Run the test, then type `exit` to finish the recording. The transcript may capture commands and visible output, including secrets. Review and sanitize it before sharing; `umask 077` limits access to the created file on systems that honor the standard umask behavior.

17. Useful command-line habits

- Use `man COMMAND`, `COMMAND --help`, and the distribution's official documentation before copying unfamiliar options.
- Use shell tab completion and `Ctrl-R` history search, then review a recalled command before running it.
- Use `printf` rather than relying on implementation-specific `echo` escape handling.
- Prefer `ss` over older `netstat` examples on current systems.
- Prefer structured output or null-delimited filenames over parsing human-formatted `ls` output.
- Never assume a command is harmless because it is one line. Check target paths, privileges, network scope, and whether it writes or deletes data.
- Never feed unreviewed data into `eval`, a shell pipeline that executes it, or a privileged command.

Command Index

Command	Recipes
`apt-cache`	10
`bash`	2, 5, 17
`column`	3
`df`, `du`, `sort`	6, 4
`diff`, `ssh`	12
`file`, `less`, `tar`	14
`getent`, `ip`	9
`grep`, `uname`	1
`journalctl`, `systemctl`	8
`python3`	15
`script`	16
`sha256sum`	13
`ss`	7

The catalog is a navigation aid; use each utility's manual page for its complete current behavior.

Source and editorial note

The recipe presentation—task first, concise command, explanation, and example—was informed by **Jason Cannon, *Command Line Kung Fu: Bash Scripting Tricks, Linux Shell Programming Tips, and Bash One-liners* (2014, ISBN 978-1-4992-2203-6)**, a book available in the author's local bookshelf. This chapter is newly written for Raspberry Pi OS, Debian, and Kali support contexts; it does not reproduce that book's text or command examples. Older one-liners need review for current utilities, safe defaults, and changed distribution behavior.

For Raspberry Pi-specific tasks, consult Raspberry Pi's official documentation. For Kali tasks, consult the official Kali documentation first. Corey P. Schultz's *Kali Linux Cookbook*, Third Edition (2025) and its publisher sample-code repository are further reading; the book and code have separate licensing terms. Do not copy book text or examples into this guide without permission.

Peace Operations Field Support

This chapter connects technical support practice with the protection, data, and coordination responsibilities that can arise in peace-support, humanitarian, and civilian-protection contexts. It is an orientation for technical support operatives—not operational training for peacekeepers, deminers, explosive-ordnance-disposal (EOD) teams, child-protection officers, or gender advisers.

The key operational lesson is to support qualified people and established procedures with dependable, secure, accessible information systems. A Raspberry Pi, database, communications link, or dashboard must not replace an accredited mine-action authority, trained child-protection specialist, or mission chain of command.

Chapters

- Mine action and explosive-hazard support
- Child protection, Women, Peace and Security, and resolutions
- Security Council resolution reference

High-impact practices covered

- **Evidence-led information management:** record source, time, confidence, consent, access limits, and changes; maintain provenance and retention controls.
- **Inclusive participation:** make information systems and risk messages accessible to women, men, girls, boys, persons with disabilities, and local communities in languages they use.
- **Privacy and do-no-harm:** minimize personal data, restrict access, avoid publishing sensitive locations or case details, and use approved referral channels.
- **Continuity and handover:** preserve support capacity and secure records through staff, service, or mission transitions.
- **Role boundaries:** support technical infrastructure without attempting explosive-hazard identification, clearance, interviewing children, or case decisions.

Read the primary sources

POTI's Mine Action and Explosive Hazard Management course is developed with UNMAS; its page describes nine lessons on treaties, mine action standards, risk education, surveys, victim assistance, and IED contexts. POTI's Child Protection for UN Military is based on United Nations training materials. Its Women, Peace and Security in Africa course was developed by UN Women in collaboration with POTI.

These courses belong to their respective authors and rights holders. This guide summarizes the public course descriptions and points to standards; it does not reproduce course text, slides, scenarios, or restricted training material.

Mine Action and Explosive-Hazard Support

This chapter applies peace-support and humanitarian mine-action concepts to **technical information support**. It does not train readers to locate, identify, move, mark, render safe, or clear mines, unexploded ordnance (UXO), explosive remnants of war (ERW), or improvised explosive devices (IEDs). Those are hazardous activities for appropriately trained and accredited specialists working under the responsible national authority and applicable standards.

1. Why mine action matters to technical support

Explosive ordnance can threaten civilians, returnees, humanitarian staff, peacekeepers, livelihoods, access to land, and public infrastructure long after active conflict has ended. Mine action is broader than clearance. The IMAS framework describes complementary work including explosive-ordnance risk education (EORE), survey and land release, victim assistance, stockpile destruction, and advocacy.

The United Nations Security Council's resolution 2365 (2017) recognized mine action's role in civilian protection and peacebuilding and called for attention to risk education, victim assistance, relevant standards, coordination, and planning. Resolution 1325 (2000) also calls attention to gender considerations in mine clearance and mine awareness programmes.

2. High-impact field practices

The practices below are **high-impact, standards-based methods**: they make decisions more evidence-based, inclusive, and sustainable without asking unqualified staff to handle explosive hazards. Their value comes from a disciplined process and trained people, not from a shortcut or a new explosive-ordnance procedure.

The transformative shift is from clearance-only thinking to a coordinated system: evidence-led land release, risk education shaped and tested with affected communities, survivor-centered referral, and information that supports accountable decisions. These practices can reduce preventable exposure and improve how scarce specialist capacity is prioritized; they do not replace trained survey, clearance, or EOD teams.

A. Start with non-technical survey and evidence quality

IMAS 08.10 describes non-technical survey (NTS) as collection and analysis of information without technical interventions, to help determine where explosive-ordnance contamination is present or absent and inform land-release decisions. It may use historical records, reports, field visits, and consultation with affected communities. NTS is not a clearance operation and does not authorize a support worker to approach suspicious items.

For a technical support operative maintaining a survey or reporting system:

- preserve the source and date of every record; distinguish observation, hearsay, and verified findings;
- keep uncertainty and confidence visible instead of silently converting reports into confirmed facts;
- use approved forms, standardized fields, time and map references, versioned records, and audit trails;
- protect raw records and coordinate corrections with the accredited survey team or national mine-action authority;
- support secure backups and offline operation where connectivity is intermittent, then use an approved reconciliation process;
- do not publicly expose hazard coordinates, reporter identities, or access routes.

NTS requires competent, trained, and in the relevant context accredited survey personnel. Decisions to cancel, reduce, or clear land belong to authorized mine-action decision-makers under the applicable national standards and “all reasonable effort” requirements.

B. Make risk education local, inclusive, and feedback-driven

EORE should be based on local needs and changing risk patterns, and designed with the intended communities. Messages may need adaptation for language, age, gender, disability, literacy, displacement, and local communication channels. Do not circulate an unverified safety poster or translate a warning mechanically and assume it is understood.

Technical support can help by:

- maintaining approved, dated versions of public safety messages;
- supporting authorized radio, print, mobile, or offline distribution plans;
- tracking reach and feedback without collecting unnecessary identity data;
- ensuring that formats are usable with low bandwidth, assistive technology, and common local devices;
- routing reports and questions to designated mine-action professionals.

If a person encounters a suspected explosive item, the support worker should not approach, touch, photograph at close range, collect components, or attempt identification. Follow the locally approved public-safety guidance, move away from danger without approaching the item, and notify the designated emergency or mine-action authority through the established channel.

C. Treat victim assistance as a coordinated, long-term responsibility

IMAS 13.10 and UNMAS guidance frame victim assistance as a rights-based, multi-sector effort involving access to emergency and continuing health care, rehabilitation, psychosocial support, education, social and economic inclusion, and relevant public services. Mine-action technical teams can help identify needs and connect people with established services; they do not replace health, rehabilitation, or social-protection providers.

Information systems should use the approved referral pathway, collect only necessary information, respect consent and confidentiality, and include survivor organizations and persons with disabilities in design and review. Data disaggregation by sex, age, disability, and other locally relevant categories can expose gaps, but only collect and share it when authorized, necessary, and protected from re-identification or stigma.

D. Coordinate nationally and across sectors

Mine action involves affected-state authorities, UNMAS, humanitarian organizations, civil society, technical operators, and communities. Information systems should make ownership, data stewardship, access, and handover responsibilities explicit. Software must not create a parallel “hazard authority” or let a user mark land as safe without the prescribed assessment and approval.

E. Use technology as a support layer—not a substitute for expertise

A Raspberry Pi or other low-cost computer may support an offline reference library, local reporting, inventory, training, or synchronization workflow. Before field use, assess physical security, power, storage durability, backup and restore, user access, data retention, secure transport, and what happens when the device is lost. Verify data formats and interoperability with the authorized mine-action information-management system.

Do not use generic image recognition, consumer GPS, crowdsourced reports, or an AI model to classify suspected ordnance or recommend a route through a suspected hazardous area. Such decisions require trained personnel and authoritative survey evidence.

3. A support workflow for mine-action information

The following workflow adapts IMAS information- and risk-management principles to ordinary technical support. The National Mine Action Authority (NMAA) or designated coordination body owns the operational rules and data authority; confirm local roles before configuring a system.

1. **Agree the information need.** Ask the responsible operations and information-management staff what decision the data supports, who provides it, who validates it, who approves sharing, and when it must be available.
2. **Use approved schemas and forms.** Record source, collection time, precision, version, and appropriate uncertainty. Do not invent fields or silently reinterpret a surveyor's finding.
3. **Protect community data.** Explain why information is being collected, obtain consent where required, minimize personal identifiers, respect confidentiality, and include community members who may otherwise be excluded.
4. **Validate with the responsible role.** Check completeness and consistency with designated information-management and operations staff. Correct errors through the approved revision process; preserve the original record and audit trail.
5. **Synchronize safely.** For offline use, document how records will be reconciled and conflicts resolved. Protect portable media and do not post raw records or hazard maps in public systems.
6. **Back up and test recovery.** Back up authorized data, restrict access to copies, and test restoration without overwriting live information.
7. **Review and improve.** Track data-quality problems, context changes, user feedback, and EORE message versions. Route operational decisions and risk acceptance to authorized decision-makers.

IMAS 05.10 assigns national information-management ownership and defines operational and information-management responsibilities. A support operative should not independently approve operational records, determine clearance priorities, authorize data sharing, or declare land safe. Do not promise that collecting a report will raise an area's priority for clearance.

EORE material release checklist for support staff

Technical support may distribute **approved** risk-education material but should not invent or alter explosive-ordnance safety messaging. Before preparing an offline pack, kiosk, SMS, web page, or shared device, check that:

- the responsible authority or qualified EORE operator approved the exact content and version;
- language, reading level, disability access, age, and local communication channels were considered by qualified staff;
- the target group and safe referral/reporting route are present and current;
- the material can be withdrawn or replaced quickly when risks or official instructions change; and
- use and feedback can be monitored without exposing personal data or sensitive hazard locations.

IMAS 12.10 calls for needs assessment, community involvement, message/material testing, monitoring, and review. The technical-support role is version control, distribution, uptime, accessibility, and privacy—not authoring unapproved safety advice.

4. Explosive-hazard encounters and role boundaries

If support work reveals a possible explosive hazard, stop the affected task, avoid further approach, warn people only in the manner specified by local procedures, and contact the designated authority. Do not:

- touch, move, open, dismantle, test, mark, or photograph an item at close range;
- ask untrained community members to inspect or identify it;
- publish precise hazard locations on an open map or public ticket;
- circulate unverified “safe route” advice;
- treat a generic online description or model-generated result as an identification.

IMAS, national mine-action standards, and mission procedures take precedence. The technical support role is to protect systems, records, communications, and authorized workflows.

5. Sources

- POTI/UNMAS, Mine Action and Explosive Hazard Management course
- International Mine Action Standards (IMAS)
- IMAS 08.10: Non-technical survey
- IMAS 12.10: Explosive ordnance risk education
- IMAS 05.10: Information management for mine action
- IMAS 07.14: Risk management in mine action
- IMAS 13.10: Victim assistance in mine action
- UNMAS, Five Pillars of Mine Action: Victim Assistance
- United Nations Mine Action Strategy 2024–2028
- UN Security Council resolution 2365 (2017)

Child Protection and Women, Peace and Security

This overview is intended for **technical support operatives** who may support systems used by peace-support or humanitarian teams. It is not a substitute for child-protection, gender, protection-of-civilians, or mission-specific training. Follow the responsible organization's safeguarding policies and the designated child-protection, gender, human-rights, and operational focal points.

1. Child protection in armed conflict

The UN Security Council's children-and-armed-conflict agenda recognizes six grave violations against children:

1. recruitment and use of children by armed forces or armed groups;
2. killing and maiming;
3. rape and other forms of sexual violence;
4. abduction;
5. attacks on schools and hospitals; and
6. denial of humanitarian access.

The Office of the Special Representative for Children and Armed Conflict explains the mandate and monitoring/reporting mechanism. Resolution 1612 (2005) established the monitoring and reporting mechanism and Security Council Working Group. Resolution 2764 (2024) stresses sustainable child-protection capacity, accountability, coordination, and continuity during peace-operation transitions.

What technical teams should do

- Protect case and incident data with strict role-based access, approved retention, secure transfer, and auditability.
- Do not put names, images, exact locations, allegations, or sensitive case details in public bug trackers, group chats, screenshots, or unencrypted removable media.
- Do not interview, photograph, identify, or collect personal details from a child as part of ordinary IT troubleshooting. Refer concerns through the approved child-protection focal point.
- Design forms and systems to collect the minimum necessary information, avoid exposing a child's identity, and work with specialists on safeguarding and consent.
- Maintain availability and safe handover of approved services during staff or mission transitions; protect credentials and audit trails.
- Ensure that digital learning or case-support systems do not expose children to surveillance, stigmatization, or unsafe contact.

UN military child-protection training is mission-specific. POTI's Child Protection for UN Military course describes scenario-based training, mandate awareness, actor coordination, and the roles of military personnel. Its training materials are sourced from the UN Peacekeeping Resource Hub under a revocable license; this book summarizes only the public course description and does not reproduce its materials or scenarios.

2. Women, Peace and Security (WPS)

The WPS agenda is commonly organized around **participation, prevention, protection, and relief/recovery**. It recognizes women's meaningful participation in peace and security decisions, prevention of conflict-related sexual violence, protection of rights, and gender-responsive relief and recovery. The agenda is not limited to women as victims; it also recognizes women's leadership and agency in prevention, mediation, peacebuilding, and community resilience.

The UN Women Global Norms and Standards page provides summaries of the ten core thematic WPS Security Council resolutions. UN Peacemaker also lists the WPS Security Council resolutions.

Applying WPS principles to technical support

- Consult affected users—including women-led organizations and groups often excluded from design—before selecting devices, reporting channels, access controls, or training formats.
- Check who can access, operate, charge, repair, and safely use each tool; identify barriers related to language, literacy, disability, safety, cost, connectivity, and social norms.
- Do not collect gender, age, location, or incident data "just in case." Collect only what is authorized and needed; restrict access and assess re-identification or retaliation risks.
- Provide confidential and accessible reporting routes; do not promise anonymity unless the technical design and operational procedure can support it.
- Make training materials usable offline and in locally appropriate formats where needed, while respecting data protection and consent.
- Include safe handover so that data, contact lists, or service access are not lost when a project or mission changes.

POTI's Women, Peace and Security in Africa course was developed by UN Women in collaboration with POTI. It covers implementation of UNSCR 1325 and subsequent resolutions, Africa-specific priorities, and roles for regional and subregional organizations. It is a course reference, not an endorsement by the course authors of this guide.

3. Resolution and mandate awareness

Security Council resolutions are not interchangeable checklists. Their operative paragraphs, addressees, scope, and legal effect depend on the exact text and context. A country or mission mandate may add requirements not captured in this thematic overview. Read the current mandate and full resolution text before configuring a system or deciding what information may be collected.

See the selected resolution reference, the UN Security Council's official resolution archive, and its WPS dashboard.

4. Further references

- POTI Child Protection for UN Military
- POTI Women, Peace and Security in Africa
- UN Special Representative on Children and Armed Conflict: mandate
- UN Women: WPS global norms and standards
- UN Security Council WPS dashboard

Selected Security Council Resolution Reference

This is a **selected thematic reading list** for mine action, children and armed conflict (CAAC), and Women, Peace and Security (WPS). It is not an exhaustive compilation of every Security Council decision relevant to a country, mission, or incident. Check the current mission mandate and the official resolution text before operational use. The brief summaries below are original orientation notes, not quotations or legal advice.

The thematic list and the public UN resolution index were reviewed on **12 October 2026**. The Security Council also adopts country- and mission-specific decisions with relevant provisions; those are intentionally not enumerated here.

Mine action and explosive hazards

Resolution	Focus	Why it matters here
S/RES/1325 (2000)	Women, Peace and Security	Calls for gender perspectives in peacekeeping and notes the need for mine-clearance and mine-awareness programmes to consider the specific needs of women and girls.
S/RES/2365 (2017)	Mine action	The first thematic Security Council resolution dedicated to mine action; recognizes the civilian threat from mines, ERW, and IEDs and calls for measures including risk education, victim assistance, standards, and early planning.

Country-specific peace-operation and humanitarian mandates can include mine-action tasks, so this thematic pair is not a complete mandate list.

Children and armed conflict

The following resolutions form the principal thematic sequence. The UN identifies six grave violations: recruitment and use; killing and maiming; rape and other sexual violence; abduction; attacks on schools and hospitals; and denial of humanitarian access.

Resolution	Main contribution (short summary)
S/RES/1261 (1999)	First Security Council resolution on children and armed conflict; places the issue on the Council's agenda.
S/RES/1314 (2000)	Develops Council attention to children's protection in conflict and peace processes.
S/RES/1379 (2001)	Requests identification/reporting of parties that recruit or use children.
S/RES/1460 (2003)	Calls for an "era of application" of child-protection norms and action by listed parties.
S/RES/1539 (2004)	Calls for time-bound action plans and development of a monitoring and reporting mechanism.
S/RES/1612 (2005)	Establishes the Monitoring and Reporting Mechanism and the Security Council Working Group on CAAC.
S/RES/1882 (2009)	Adds killing and maiming and rape/sexual violence as listing triggers for parties.
S/RES/1998 (2011)	Adds attacks on schools and hospitals as listing triggers.
S/RES/2068 (2012)	Reaffirms action against persistent perpetrators and implementation of commitments.
S/RES/2143 (2014)	Addresses military use of schools and child-protection training for peacekeepers.
S/RES/2225 (2015)	Adds abduction of children as a listing trigger.
S/RES/2427 (2018)	Provides a broader prevention, protection, and reintegration framework across the conflict cycle.
S/RES/2601 (2021)	Focuses on protection of education and continuation of education in armed conflict.
S/RES/2764 (2024)	Emphasizes sustainable child-protection capacity and coordination during UN peace-operation transitions.

The six-violation list and country situations evolve through Secretary-General reports, action plans, and mission mandates; check the official CAAC mandate and resources.

Women, Peace and Security

UN Women identifies ten core thematic WPS resolutions. Resolution 2538 (2020) is a related, dedicated decision on women in peacekeeping; it is included separately because reference lists differ in whether they count it as part of the core ten.

Resolution	Main contribution (short summary)
S/RES/1325 (2000)	Foundational WPS framework: participation, gender perspectives, protection, and women's role in peacebuilding.
S/RES/1820 (2008)	Recognizes conflict-related sexual violence as a matter of international peace and security.
S/RES/1888 (2009)	Strengthens UN leadership and response mechanisms on sexual violence in conflict.
S/RES/1889 (2009)	Focuses on women's participation in peacebuilding and monitoring implementation.
S/RES/1960 (2010)	Establishes further monitoring and reporting measures on conflict-related sexual violence.
S/RES/2106 (2013)	Emphasizes accountability and implementation of obligations on sexual violence.
S/RES/2122 (2013)	Calls for consistent integration of WPS and women's participation in peace processes.
S/RES/2242 (2015)	Establishes the Informal Experts Group and advances implementation and participation.
S/RES/2467 (2019)	Strengthens a survivor-centred approach to preventing and responding to conflict-related sexual violence.
S/RES/2493 (2019)	Calls for full implementation of previous WPS resolutions and meaningful participation.
S/RES/2538 (2020)	Dedicated resolution calling for women's full, effective, and meaningful participation in UN peacekeeping.

Related protection-of-civilians references

For broader civilian-protection context, also consult S/RES/1265 (1999), S/RES/1296 (2000), S/RES/1674 (2006), and S/RES/1894 (2009). For the safety and security of humanitarian personnel and UN/associated personnel, see S/RES/2730 (2024). These are not substitutes for the applicable country mandate or humanitarian law.

How to use this reference

1. Read the operative paragraphs of the resolution, not only a summary.
2. Check the mission mandate, host-state framework, and current UN policy that apply to the location and role.
3. Distinguish Security Council resolutions from General Assembly resolutions, treaties, International Mine Action Standards, and training-course material; they have different status and scope.
4. Record the document identifier and date in support plans. Review amendments and later decisions before a deployment.

Official starting points

- UN Security Council resolutions archive
- Security Council WPS dashboard
- UN Women: WPS norms and standards
- UN Office of the Special Representative for Children and Armed Conflict

Informatics, Public Health, and Corporate Accountability

Informatics is not only about building software. It includes how data is collected, who controls it, whose interests shape a system, what evidence supports its decisions, and who bears the consequences when it fails. This chapter connects that perspective to William Majanja's **Muainishi** application series and to public-health and agricultural claims.

The guide will critically examine documented harms, conflicts of interest, market incentives, privacy abuses, and regulatory failures. It will not present the unsupported blanket claim that "big corporations are killing people with genetically engineered crops." Genetically engineered crops are not one product or exposure, and a company's profit motive alone does not prove a specific causal effect or intent to cause harm. Strong criticism is most useful when a claim identifies the actor, product, exposure, evidence, affected population, and accountability finding.

1. The Muainishi application series

These are repository-described prototypes and research/learning concepts—not validated medical products:

Application	Repository-described focus	Support and governance questions
Muainishi Wa Jenomu	Browser-based genomics exploration, classification, and visualization.	Its README describes Gemini-backed analysis while also making a browser-local privacy claim. Verify actual outbound requests and data handling before use; do not upload identifiable genomic data without authorization, consent, and a privacy review.
Muainishi Wa Viungo	A concept for transfusion, transplant, and related biomedical information.	Validate clinical claims, data provenance, access roles, and external AI/API transfers. Use synthetic records in demonstrations.
Muainishi Wa Saratani	AI-assisted oncology classification and information support.	Its README describes generated classifications and treatment information. Do not treat outputs as diagnosis or treatment recommendations; use only synthetic data unless a qualified institution has approved the system and its data protections.

A practical assessment checklist

1. **Data source:** What dataset is used? Is it lawfully obtained, licensed for this purpose, representative, and documented with version and provenance?
2. **Consent and purpose:** Were people told how their data would be used, where it would be sent, how long it would be retained, and how to seek correction or deletion?

3. **Data flow:** Does processing stay on the device, or does a browser call a cloud API? A web interface running in a browser does not prove that uploaded data stays local.
4. **Credentials:** Does a client-side app expose an API key? Client-side keys are visible to users and must not be treated as confidential credentials.
5. **Validation:** Was a model assessed against an appropriate independent reference set? Are error rates, missing groups, uncertainty, and limitations visible?
6. **Human review:** Which qualified professional is responsible for evaluating the result? Can the system be overridden and audited?
7. **Impact:** Could an inaccurate label create stigma, denied care, inappropriate treatment, or exposure of a person's identity or ancestry?

Public repositories can be useful design studies, but promotional descriptions, model-generated confidence scores, or polished interfaces do not establish scientific validity.

2. Corporate incentives and documented public harm

Large organizations can provide valuable products and services; their size or profit motive alone does not establish wrongdoing. Yet documented cases show that corporate decisions can cause or amplify large-scale public-health harms: WHO reports more than seven million annual deaths from tobacco, and DOJ records Purdue Pharma's admissions about marketing opioids despite diversion concerns and paying kickbacks to increase prescriptions. WHO also describes a fundamental conflict between tobacco-industry and public-health interests. Harm can arise when sales incentives, market power, opaque practices, weak oversight, or externalized health and environmental costs outweigh safety and accountability. Evaluate each case from records and evidence rather than assigning one motive to an entire industry.

Documented examples—not claims about every company

- **Pharmaceutical marketing and opioid diversion:** the US Department of Justice records Purdue Pharma's 2020 guilty pleas to three federal felonies, including conspiracies involving defrauding the DEA/FDA and anti-kickback laws. DOJ's case page summarizes Purdue's admissions about marketing and diversion controls. This is a specific, documented case; it is not proof that every pharmaceutical company behaves the same way or that every medicine is harmful.
- **Technology and personal-data control:** the US Federal Trade Commission imposed a $5 billion penalty and new privacy restrictions on Facebook in 2019 to resolve charges that it violated a prior FTC privacy order by deceiving users about their control of personal information. In a separate matter, the FTC issued an opinion finding that Cambridge Analytica engaged in deceptive practices to collect Facebook users' data, followed by a final order. These records document serious privacy harms; they do not establish that all technology companies cause physical injury or death.
- **Tobacco:** the World Health Organization reports that tobacco causes multiple cancers and more than seven million deaths annually worldwide. WHO also identifies a fundamental conflict between tobacco-industry interests and public-health interests. This is a well-established product-category harm; use product-specific evidence when comparing other industries.

The point of these examples is to show **how to investigate accountability**: identify a particular firm and product, establish what was known and when, examine the regulatory or court record, distinguish allegations from admissions or findings, and follow the effects on people and communities.

3. Genetically engineered crops, pesticides, and cancer claims

Avoid treating “GMO,” a crop trait, a pesticide, a food residue, a seed contract, and an agricultural business model as if they were the same thing.

- WHO’s food genetically modified Q&A explains that safety evaluation is **case by case** because genetically modified organisms differ in traits and methods. The page is dated 2014; check current national assessments and local regulation rather than presenting it as the final word for every product.
- The US National Academies’ 2016 consensus report reviewed evidence on health, environmental, agronomic, economic, and regulatory effects. For the commercial GE crops it reviewed, it found no substantiated evidence that foods were less safe than comparable non-GE foods; it also described varied agronomic outcomes, including insect resistance, herbicide-resistant weeds, and the need for sound resistance-management practices. Its findings cover the evidence and products available to that report—not every future crop or farming practice.
- A herbicide-tolerant crop does not make the herbicide and the genetic trait identical. Assess the specific active ingredient, formulation, exposure route, dose, use pattern, residues, worker protections, local environment, and crop separately.
- IARC classified **glyphosate** as Group 2A (“probably carcinogenic to humans”) in 2015, based on its evaluation of the available evidence. IARC explicitly distinguishes **hazard identification** from estimating the **risk under a particular exposure scenario**. National regulators may conduct separate risk assessments and reach different conclusions. Neither result justifies a blanket statement that genetically engineered crops as a class cause cancer.

Questions about seed patents, farmer choice, pesticide sales, market concentration, or environmental effects should be examined with agricultural data, local law, contract terms, independent research, and affected farmers’ accounts. Where evidence is incomplete or contested, say so.

4. A reproducible method for investigating harm

Use this checklist before publishing a health, safety, or corporate-accountability claim:

1. **State a narrow question:** name the product or practice, company, place, time period, exposure route, and outcome.
2. **Separate claim types:** distinguish a hazard from a measured risk, correlation from causation, an allegation from an admission or finding, and a theoretical mechanism from observed harm.
3. **Use primary evidence:** regulatory assessments, court judgments or pleas, peer-reviewed studies, official health guidance, audit records, and transparently documented data.
4. **Compare sources:** look for systematic reviews, independent replication, methods, sample size, uncertainty, confounders, effect sizes, and funding/conflict disclosures.
5. **Track jurisdiction and date:** approvals, limits, and litigation findings vary by country and change over time.
6. **Represent people respectfully:** do not publish identifiable health, genomic, disability, or location data without a lawful basis and appropriate consent.
7. **Report limits and corrections:** make citations usable, distinguish verified from unverified statements, and update the page when evidence changes.

A profit motive can help explain an incentive; it does not by itself prove that a specific company intentionally caused a specific harm. Intent requires case-specific evidence such as internal records, admissions, regulatory findings, or judicial findings.

Other sectors to examine with the same method

Tobacco, fossil-fuel pollution, mining and occupational exposures, industrial chemicals, and food marketing can all raise serious public-interest questions. Investigate a named substance, process, company, exposure, and affected community rather than treating an entire sector as a single actor. Cancer-hazard classifications, exposure risk assessments, court findings, and public-health surveillance answer different questions and should not be conflated.

5. How Raspberry Pi and open Informatics can help

In a properly governed project, a Pi can support a local/offline reference library, field data entry, transparent inventory, or community reporting workflow. It is not automatically private or secure because it is small or locally hosted. Apply the same controls as any information system:

- collect the minimum data needed for a defined purpose;
- use access roles and protect devices, storage, backups, and exported reports;
- record data source, collection time, schema/model version, and uncertainty;
- test restore and offline synchronization before field deployment;
- review cloud API use, data residency, retention, consent, and credentials;
- publish aggregated or de-identified information only after checking re-identification risk and authority to disclose it.

For health or agricultural decisions, maintain a qualified human decision-maker, an appeal/correction path, and an explicit statement of what the software has and has not been validated to do.

References

- WHO: Food, genetically modified
- National Academies: Genetically Engineered Crops—Experiences and Prospects
- IARC: Glyphosate evaluation
- IARC: Hazard identification versus risk assessment
- WHO: Tobacco and nicotine
- US DOJ: United States v. Purdue Pharma L.P.
- US FTC: Facebook privacy penalty and restrictions
- US FTC: Opinion and order against Cambridge Analytica

Scope note: this chapter supports informed investigation and responsible data practice. It is not medical, legal, agricultural, or regulatory advice.

Zero-Dependency Quizzes and CTF-Style Challenges

This self-study track borrows the **progressive level, prompt, hint, flag, and answer-key** pattern of capture-the-flag learning, including William Majanja's HydraCTF. The challenges and answers on this page are newly written for this guide; no HydraCTF code, database, flag, level text, or challenge content is copied.

Zero-dependency rules

- No CTF server, internet connection, package install, Kali tool, account, or external dataset is needed.
- Challenges use only information in the prompt, the guide, and basic reasoning. Optional command-line approaches use standard Linux tools already covered in the book.
- Flags are learning tokens, not credentials or access tokens. They do not authenticate to a service.
- Cryptography levels use **toy encodings and concepts**. They do not teach breaking real encryption or attacking systems.
- Work on the worksheet offline if desired. The answer key contains spoilers.

How to play

1. Read one level and write down the safest correct decision or decoded toy string.
2. Submit the exact case-sensitive `FLAG{...}` token shown by your result.
3. Record a one-sentence explanation. A flag without a correct explanation does not demonstrate understanding.
4. Check the answer key only after attempting the level.

Base Levels — Field Support

BASE-0 — Identify the box

Scenario: A device reports `ID=debian`, `VERSION_ID=13`, and `uname -m=aarch64`. A colleague asks you to add Kali repositories so `kali-linux-default` can be installed.

Task: State the safe next step. Which OS repository must remain configured?

Hint: Hardware architecture does not determine which distribution repositories are safe to mix.

Flag format: `FLAG{...}`

BASE-1 — Protect the data before recovery

Scenario: A Raspberry Pi will not boot after power was disconnected during a write. Its storage may hold the only copy of a community project's data.

Task: Choose the first response that minimizes data loss. Do not assume the storage is empty or irrecoverable.

Hint: Record the symptom and verify supply/boot indicators for the exact board before re-imaging.

Flag format: `FLAG{...}`

BASE-2 — Diagnose one layer at a time

Scenario: `ip -brief address` shows `wlan0 UP 192.168.1.42/24`; `ip route` shows a default route via `192.168.1.1`; `getent hosts example.org` returns no address.

Task: Identify what to investigate next without scanning other network devices.

Hint: The Pi has a link, an address, and a route. Name the next layer.

Flag format: `FLAG{...}`

BASE-3 — Protect sensitive records

Scenario: A public support ticket contains a child's name, a case narrative, and exact coordinates for an explosive-hazard report.

Task: State the safe response and the proper route for handling the record.

Hint: Do-no-harm, data minimization, and the organization's designated protection channel apply to technical support too.

Flag format: `FLAG{...}`

Quiz Levels — Knowledge Check

Each quiz is worth one point. Answer without consulting the key; use the linked chapter to review any missed concept. No software is required.

QUIZ-0 — Who handles a suspected explosive item?

Choose one:

A. A technical support operative takes a close-up photo and identifies it from a database. B. Nobody approaches or handles it; follow local safety procedure and report through the designated trained authority. C. A community member moves it away from a path.

QUIZ-1 — Which resolution is the thematic Security Council reference on mine action?

Choose one:

A. S/RES/2365 (2017) B. S/RES/2493 (2019) C. S/RES/2764 (2024)

QUIZ-2 — What is the safe treatment of a child-protection report in a support ticket?

Choose one:

A. Include the child's name and exact location so the ticket can be searched publicly. B. Send the case details to the technical mailing list for awareness. C. Minimize identifying details and use the organization's restricted, designated child-protection channel.

QUIZ-3 — What does meaningful WPS participation require in support-system design?

Choose one:

A. Consult only mission leadership and assume tools work equally for every user. B. Consult affected women and relevant groups; address access, safety, and privacy barriers. C. Collect gender and precise location data by default, whether or not needed.

QUIZ-4 — How should you assess the claim “genetically engineered crops cause cancer”?

Choose one:

A. Treat every crop trait, pesticide, exposure, and food as one identical hazard. B. Identify the specific crop/trait, product and exposure, consult current case-by-case assessments, and distinguish hazard evidence from exposure-specific risk. C. Assume no engineered crop or associated agricultural practice can ever cause harm.

QUIZ-5 — Does a company’s profit motive prove that it intended to cause a particular harm?

Choose one:

A. Yes; a financial incentive alone proves intent and causation. B. No; examine the specific evidence, product, exposure, regulatory record, and any admissions or findings before claiming intent or causation. C. Corporate conduct can never be investigated from public records.

Crypto Flag Levels

CRYPTO-0 — ROT13 is encoding

Ciphertext: `SYNT{GUR_RAPBQRQ_ZRFFNTR_VF_ABG_FRPERG}`

Task: Rotate alphabetic characters by 13 places. Keep braces and underscores unchanged. What does the flag say?

Hint: A becomes N; N becomes A. This is a reversible substitution, not secure encryption.

Flag format: The decoded string itself.

CRYPTO-1 — Caesar shift is a toy

Ciphertext: `IODJ{FDHVDU_LV_QRW_HQFUBSWLRQ}`

Task: Each alphabetic character was shifted forward by 3. Shift backward by 3 to recover the flag.

Hint: I becomes F. Preserve capitalization and punctuation.

Flag format: The decoded string itself.

CRYPTO-2 — Hashes are not ciphertext

Scenario: A support log contains a SHA-256 digest of a credential. Someone asks you to “decrypt the hash” to recover the password.

Task: Choose the technically correct, authorized support response: decrypt it, publish it for lookup, or use the approved credential reset/verification process while protecting the digest.

Hint: Hashing and encryption have different purposes; do not expose real credential material in a challenge.

Flag format: `FLAG{...}`

Debrief and next steps

The learning objective is sound judgment, not speed. For system support, continue with the field troubleshooting playbooks. For peace-operations context, read mine-action support and child protection/WPS. For cryptography vocabulary, read CPIP- architecture and principles and its caveats. Try other CTFs only in their intended, authorized environment.

Inspiration and attribution

HydraCTF is a separate project by William Majanja, described in its repository as a CTF for hackathons and societal scenarios. This book uses only the broad educational idea of progressive levels and flags. Its challenge content and answer key are original to this guide.

Challenge Answer Key

Spoilers: attempt the levels in Challenges before opening this page.

Base Levels — Field Support

BASE-0

Flag: `FLAG{KEEP_DISTROS_SEPARATE}`

The machine identifies as Debian 13. Keep Debian repositories configured; use supported Debian packages or a separate Kali VM/device for Kali-only packages. Do not add Kali sources to Debian.

BASE-1

Flag: `FLAG{PRESERVE_BEFORE_REIMAGE}`

Record the symptom and hardware/OS details, check the model-specific power and boot indicators, and preserve the original storage before any re-image attempt. Re-imaging overwrites the target media.

BASE-2

Flag: `FLAG{DIAGNOSE_IN_LAYERS}`

The interface is up, has an address, and has a default route. Investigate name resolution using the approved resolver configuration and authorized diagnostics. Do not infer that the whole network is down or scan unrelated devices.

BASE-3

Flag: `FLAG{DO_NO_HARM_WITH_DATA}`

Remove or restrict the public exposure through the authorized support process, preserve only what is necessary, and route the case through the organization's designated child-protection channel. Do not repost the identifying information in a ticket.

Quiz Levels

- **QUIZ-0:** B. A support operative does not inspect or identify an explosive hazard; follow local safety procedures and report it to qualified personnel.
- **QUIZ-1:** A. S/RES/2365 (2017) is the thematic mine-action resolution.
- **QUIZ-2:** C. Minimize case data and use the designated restricted child-protection pathway.
- **QUIZ-3:** B. Meaningful participation and safety/accessibility barriers matter; avoid unnecessary sensitive data collection.
- **QUIZ-4:** B. Genetic engineering is a method, not one uniform exposure; assess a specific product and evidence case by case, and separate hazard from risk at a particular exposure.
- **QUIZ-5:** B. Profit can create incentives, but proving a specific harm or intent requires case-specific evidence. Purdue is a documented example with its own legal record, not a basis for claims about every pharmaceutical company.

Crypto Flag Levels

CRYPTO-0

Flag: `FLAG{THE_ENCODED_MESSAGE_IS_NOT_SECRET}`

ROT13 shifts each letter by 13 places. It is an encoding/substitution exercise and provides no confidentiality.

CRYPTO-1

Flag: `FLAG{CAESAR_IS_NOT_ENCRYPTION}`

Shift each letter backward by three positions. Caesar substitution is a toy cipher and must not be used to protect data.

CRYPTO-2

Flag: `FLAG{HASH_IS_NOT_CIPHERTEXT}`

A cryptographic hash is not decrypted. Use the approved credential reset and verification process; do not disclose the digest or attempt recovery of a real credential outside an authorized process.

CPIP-: Coffee Pot Internet Protocol

CPIP- is William Majanja's independent **Coffee Pot Internet Protocol** project. It is not an official Kali package or a preinstalled Kali tool. It builds on the deliberately whimsical HTCPCP standards for coffee and tea and describes additional networking, messaging, dashboard, and defensive features.

This chapter is a reader's guide to understanding and evaluating the project. It distinguishes the repository's stated design goals from independently verified properties. The documentation checked for this edition is the project's public `master` branch as viewed on **12 October 2026**; the repository's README, package metadata, and release notes show version drift. For reproducibility, choose and record a specific reviewed commit before following version-dependent steps.

What "zero dependency" means here

This book is static Markdown and does not need a site generator or extra documentation packages. CPIP- itself is **not zero-dependency**: its current `pyproject.toml` declares Python 3.10+ plus required Python packages, including `cryptography`, `numpy`, and `pqcrypto`; setuptools is its build backend. The TUI, rich CLI, and HSM integrations add optional dependencies. Check the exact revision's `pyproject.toml` before installing.

The project's stated design goals

The project's Blackpaper describes goals including:

- following the HTCPCP/HTCPCP-TEA vocabulary;
- operating a logical mesh over multiple transport types;
- providing an operator-facing CLI and dashboard;
- experimenting with cover traffic and active network defense;
- combining classical cryptographic algorithms with a research post-quantum component.

These are **project-stated goals**, not evidence of effectiveness, safety, privacy, standards conformance, or security review. A design goal such as "plausible cover traffic" does not make traffic undetectable. A tool's feature label does not prove the feature works as claimed.

Guide to this chapter

- Understand the architecture, protocol, and project principles
- Set up and try a local-only instance
- Configure, operate, and assess security

Safe starting point

Begin with a source review and a disposable, isolated environment. Do not expose a fresh instance to a network: project documentation describes defaults that may bind beyond loopback and enable services or discovery. For the first walkthrough, keep the service on `127.0.0.1`, disable mesh and optional transports, and use non-sensitive test data. Do not run deployment scripts as root without reading and understanding every change they make.

> **Research project, not an assurance claim:** the repository describes some features as FIPS-aligned or defensive, but its own security policy says CPIP is not a FIPS-validated cryptographic module. Its custom Kyber/ML-KEM variant is explicitly non-FIPS. No independent security audit is established by the material reviewed for this guide.

Primary sources

- CPIP- repository
- README and quick start
- Quick Start
- Usage examples
- Security policy
- Blackpaper
- RFC 2324: HTCPCP
- RFC 7168: HTCPCP-TEA

CPIP- Architecture and Principles

1. The protocol joke and the project

RFC 2324 defines the Hyper Text Coffee Pot Control Protocol (HTCPCP); RFC 7168 extends it to tea. CPIP- uses that coffee-and-tea vocabulary as the basis for a software project. Despite the humorous protocol, the repository describes real network listeners, peer messaging, cryptographic operations, optional transports, a web interface, and active network defenses. Treat it as network software, not as a harmless toy merely because its protocol is playful.

The project is independent of the Kali distribution. The word “CPIP” is also used by unrelated projects; here the trailing hyphen identifies William Majanja’s Coffee Pot Internet Protocol repository.

2. Architecture as described by the project

The repository describes these main layers:

Layer	Project-described role	Questions to verify for a chosen revision
HTCPCP service	Coffee/tea request handling and status	Which methods, paths, and status codes are actually implemented?
HTTP/API and dashboard	Local status, configuration, controls	Which endpoints require authentication? Which actions change state?
Node and mesh logic	Peer discovery, routing, message queues	How are peers authenticated? What metadata is exposed?
Transports	LAN UDP and optional satellite, radio, or mobile links	Which components are compiled/enabled, and what hardware or service is required?
Cryptography	Message protection, identity, and key derivation	Which code path is used? What key is trusted, where is it stored, and what has been independently reviewed?
Defensive features	Probe heuristics, rate limits, and policy toggles	What are the false-positive modes, and what is actually blocked?

The table summarizes the project documentation; it is not an architecture audit. Verify all ports, defaults, authentication behavior, data storage, and feature availability against the exact source revision you intend to run.

3. Reading the design goals critically

RFC fidelity

Use the RFCs as protocol references, not as a security certification. An implementation can use valid method names while still having implementation bugs or unsafe network defaults. Check the relevant RFC section and test behavior against the current code.

Multi-transport resilience

Multiple paths can improve availability, but every enabled transport adds configuration and attack surface. Radio, cellular, satellite, and relay behavior may expose location, network identifiers, peer relationships, timing, or other metadata. Enable only transports you own or are authorized to use, and document hardware, service, and regional regulatory requirements.

Cover traffic and privacy

The project describes encoding data in protocol additions and generating cover traffic. Cover traffic is not a guarantee of anonymity or resistance to traffic analysis. Endpoints, timing, packet sizes, identifiers, radio emissions, DNS requests, and the surrounding network may remain observable. This guide explains the design at a conceptual level and confines any demonstration to an isolated lab.

Active network defense

The project describes heuristic probe detection, HTTP 418 responses, and IP blacklisting. Heuristics can block legitimate clients, including scanners used by an authorized administrator. Treat these as experimental controls: assess logs, test false positives on localhost or a private lab, and do not deploy them as a substitute for a firewall, authentication, monitoring, or an incident-response plan.

Cryptographic claims

- “Uses an algorithm named in a FIPS publication” does not mean the software or cryptographic module is FIPS validated.
- A startup self-test is useful diagnostics but is not a certification.
- The project labels its modified Kyber/ML-KEM implementation as non-FIPS and experimental. Do not rely on it for regulated or production security requirements.
- A cryptographic primitive does not compensate for weak key management, unauthenticated peer identity, incorrect protocol composition, insecure defaults, or implementation defects.
- Independent review of a particular source revision is required before making assurance claims.

For standards background, use the primary NIST publications and the project’s own security policy. Do not repeat stronger assurances than those sources justify.

4. Threat model and data questions

Before a lab or deployment, write down:

1. Which systems and networks are in scope, and who authorized the test?
2. What information can be observed by each transport, relay, peer, web browser, and operating system?
3. Where are keys, message history, logs, and configuration stored? How are they backed up and deleted?
4. Which component listens on each interface and port? Does it have an authentication boundary?
5. What happens if a peer is malicious, a key is copied, a transport fails, or a defense feature blocks a legitimate administrator?
6. How can the operator stop the service and restore the previous system state?

The project’s own threat model is useful input, not a substitute for one specific to your deployment.

5. Source/version discipline

The project's public documentation has version mismatches: its README and `pyproject.toml` describe a newer version than some release notes and deployment examples. Do not mix commands, container tags, or configuration tables from different versions.

For a reproducible review, record:

- repository URL and commit hash or signed release tag;
- date reviewed and Python version/architecture;
- dependency lock or resolved package versions;
- enabled transports and environment variables;
- tests run and their results;
- deviations from upstream instructions.

Prefer source links pinned to a commit in a published walkthrough. A link to `master` is convenient but changes over time.

CPIP-: Local-Only Setup

This walkthrough is designed for an **isolated, disposable test environment**. It is not a production deployment guide. It does not demonstrate covert operation, evasion, scanning of third parties, or radio transmission.

1. Check prerequisites and source

CPIP-'s current package metadata requires Python 3.10 or later and declares required dependencies, including `cryptography`, `numpy`, and `pqcrypto`. Its build backend uses setuptools. Optional terminal UI and HSM features add more dependencies. ARM64 availability depends on the exact Python and package versions selected; check whether compatible wheels exist before attempting an install.

Review the repository's README, `pyproject.toml`, `SECURITY.md`, and install/deployment scripts. Choose a specific tag or commit. Do not assume that the `master` branch has been audited or that its version badge matches every included manifest.

The following commands are a **generic Python project workflow**, not a claim that a particular CPIP- commit has been executed on this machine:

```
git clone https://github.com/WilliamMajanja/CPIP-.git
cd CPIP-
# Check out a reviewed tag or commit before installing.
python3 -m venv .venv
. .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install .
```

Installing `.` resolves the package metadata and declared dependencies for the checked-out revision. Do not use `sudo pip`, install dependencies globally, or paste unreviewed shell installers into a privileged shell. If a required package is unavailable for the target architecture, stop and resolve compatibility rather than weakening the environment or silently substituting versions.

2. Start a local-only service

The project documentation describes the following variables, but defaults can change. Inspect the selected source revision to confirm they are recognized and behave as expected before starting the service.

For an initial local test, keep the listener on loopback and disable external transports and mesh features. An example configuration is:

```
CPIP_BIND=127.0.0.1 \
CPIP_SSL=0 \
CPIP_MESH=0 \
CPIP_SAT=0 \
CPIP_RADIO=0 \
CPIP_MOBILE=0 \
CPIP_COVERT=0 \
./server.py
```

The `CPIP_BIND` and feature variables are documented by the project; verify their names and defaults for your pinned version. If the server cannot be confirmed as loopback-only, do not continue with a shared network connection. Run as an ordinary user in a disposable virtual machine or isolated lab, never as root for this walkthrough.

3. Verify a basic local request

In a second terminal on the same machine, use the project's documented dashboard URL and a simple RFC-style request. The examples below use plain HTTP only because this local walkthrough explicitly disables TLS and binds to loopback:

```
curl --max-time 5 -i http://127.0.0.1:4180/
curl --max-time 5 -i -X BREW http://127.0.0.1:4180/coffee
```

Open `http://127.0.0.1:4180/dashboard` only if the pinned version serves that route. Do not forward the port from a router or expose it through a public tunnel. Stop the server with `Ctrl-C` and verify that its listener has exited.

If a request fails, inspect the server output, verify that the selected port is free, confirm the configured bind address, and consult the documentation for the same commit. Do not respond to a failure by enabling all network transports or opening firewall ports.

4. CLI and dashboard

The repository documents a Bash CLI named `cpip`, a web dashboard, and optional TUI support. Commands and installed paths vary by revision and installation method. Start with read-only status/version commands from the pinned README or `./cpip --help`; check every command against the version actually installed.

Keep controls on localhost. The project's security policy states that mutating API calls require HMAC authentication by default, but this must be confirmed in the code path and configuration of the chosen revision. Do not place tokens or secret keys in screenshots, shell history, Git, issue reports, or public logs.

5. Optional transports and hardware

Do not enable optional transports as part of the first-run test. Each needs a separate review:

- **LAN mesh:** restrict to a lab network you administer; document discovery and broadcast behavior.
- **Radio/LoRa/TNC:** require compatible hardware and legal frequency, duty-cycle, power, and licensing checks for your location. A "simulation" mode must be confirmed not to transmit.
- **Mobile/cellular:** review modem interfaces, carrier terms, identifiers, and location/telemetry exposure.
- **Satellite or relay paths:** verify service ownership, destination, egress policy, and metadata exposure.

This guide does not provide instructions for hiding activity from network operators, bypassing controls, or using covert channels on systems or networks without explicit authorization.

6. What has and has not been tested

This repository documents the procedure but does not claim the CPIP- application was installed or exercised by this book's validation. The authoring desktop is Debian 13 Trixie, ARM64, Wayland, KDE Plasma 6.3.6. That fact does not substitute for testing a particular CPIP- release, and it does not establish compatibility with Kali. Report results only for the exact commit, hardware, Python, dependencies, and configuration tested.

CPIP-: Operations and Security Review

This chapter helps an authorized operator make an informed decision about a CPIP- deployment. It deliberately separates project documentation from verified behavior. It is not an endorsement for production, high-risk communications, or regulated use.

1. Configuration inventory

The project documents environment variables for the listener, TLS, mesh, covert messaging, satellite/radio/mobile transports, policy controls, persistence, and cryptography. Use the environment-variable table from the **same pinned commit**; defaults and names have changed across documentation versions.

Before starting, create a written configuration inventory:

Area	Record
Identity	Commit/tag, checksum, source of the checkout
Runtime	Python version, architecture, resolved dependencies
Listener	Bind address, port, TLS mode, exposed routes
Authentication	Enabled mechanisms, secret generation and storage, rotation plan
Transports	Explicitly enabled transports, interfaces, peers, ports, hardware
Persistence	Paths, permissions, retention, backup and deletion procedure
Logging	Location, sensitive fields, retention and access controls
Recovery	Stop procedure, restore plan, known-good system state

Start with optional services and transports disabled. Enable only one feature at a time after reviewing its code, documentation, permissions, and network effects.

2. Network exposure and authentication

The project documentation describes an HTTP service, dashboard, API, and network transports. Its documented bind default may listen beyond loopback; never assume that a development server is local-only. Confirm the actual listening socket from the process and the host's network tools before connecting another device.

Operational rules:

1. Keep a learning instance bound to `127.0.0.1`.
2. Do not put the dashboard/API directly on the public internet.
3. Confirm authentication is enforced on **every** mutating route, including dashboard routes and optional services.
4. Use unique, non-default secrets and protect environment files with restrictive permissions.
5. Limit inbound connections at the host firewall to the minimum required for a lab.

6. Do not interpret a successful TLS handshake or security-header list as proof of a secure deployment.

The repository describes auto-generated self-signed certificates and sample `curl -k` commands. `-k` disables certificate verification. It is acceptable only as an explicit loopback diagnostic when you understand that server identity is not being verified; do not copy it into production or general examples. For a real deployment, configure a trusted certificate and verify it normally.

3. Keys, messages, and persistence

Before storing real data, inspect the pinned source for key generation, peer identity verification, key derivation, key rotation, encrypted persistence, backups, deletion, swap/core-dump behavior, and failure handling. Do not assume that a setting called “encrypted” or “secure wipe” provides a guarantee without examining implementation and threat model.

Use disposable keys and synthetic messages in a lab. Do not reuse SSH, account, API, or production secrets. Do not put a real person’s health, election, location, or operational data into test messages. Decide how to revoke peers and remove stored data before enabling persistence.

4. Crypto language: precise claims

- AES-GCM, ECDSA, ECDH, HKDF, and HMAC are names of algorithms or constructions; their use does not certify an application.
- FIPS publication references describe standards. **CPIP- is not thereby FIPS 140-2/3 validated.** Its own security policy says the application is not a validated module.
- Project-described startup self-tests show that selected code paths passed known-answer/round-trip checks; they do not establish a formal certification or system-level security.
- The project’s modified Kyber/ML-KEM component is labeled non-FIPS and experimental. It should not be used to satisfy FIPS 203 requirements.
- A hybrid construction’s claimed security property depends on exact composition, authentication, implementation, and key management; do not turn a project statement into a guarantee.

For compliance or high-assurance use, obtain an independent evaluation of the exact release and cryptographic module. Do not base a deployment decision solely on this guide or the project’s marketing language.

5. Active defense and diagnostics

The project describes probe scoring, request blocking, IP blacklisting, and network-diagnostic endpoints. These mechanisms are heuristic and may have false positives, lock out legitimate operators, or generate traffic outside the host. Test on localhost or an isolated network with explicit authorization. Keep logs and a rollback route. Do not scan, probe, block, or alter systems outside your scope.

Disable a feature using the documented control path only after confirming the selected version, required authorization, and effect. “Temporary” runtime switches may not persist after a restart; compare live state with the environment and service configuration before relying on a setting.

6. Deployment modes

The project repository includes or describes standalone, systemd, Docker, Kubernetes, Raspberry Pi, and radio setups. These are distinct deployment modes with different privileges, networking, and persistence. Do not combine snippets from different versioned examples.

Before using a deployment script or manifest:

- inspect every file it installs or changes;
- verify container image names and tags against the pinned release;
- check service user, capabilities, mounts, ports, bind address, and restart behavior;
- confirm secrets are not committed or printed;
- check firewall changes and rollback instructions;
- deploy first in an isolated test system and capture the before/after state.

In particular, do not run an installer as root merely because the README suggests `sudo`. Review the script and prefer an unprivileged, contained test when possible.

7. Troubleshooting checklist

- **Server will not start:** verify Python version, installation environment, dependency compatibility for the architecture, and syntax against the pinned revision.
- **Port already in use:** inspect the existing listener; do not change the bind to all interfaces as a workaround.
- **Dashboard unavailable:** verify local bind, configured port, server route, and browser certificate behavior.
- **Authentication failure:** use the same-version security documentation; never disable authentication to make a network-facing test pass.
- **Transport unavailable:** confirm the required hardware, interface, permissions, optional dependency, and regional rules. Keep other transports disabled while diagnosing.
- **Unexpected network traffic:** stop the service, preserve logs/configuration, isolate the test host, and compare the actual environment with the recorded inventory.

8. Shutdown and cleanup

Stop the foreground server with `Ctrl-C`; for managed services, use the matching service manager command only after checking the service name. Verify listeners have stopped. Remove temporary test keys, messages, and generated files from the disposable lab; retain sanitized logs only if useful for the test record. Restore any firewall, service, or environment changes documented in your pre-test inventory.

9. References

- CPIP- security policy
- CPIP- configuration and deployment documentation
- NIST FIPS publications
- Debian security documentation
- Kali documentation